

Received 9 September 2024, accepted 11 November 2024, date of publication 18 November 2024, date of current version 24 February 2025.

Digital Object Identifier 10.1109/ACCESS.2024.3501358

 SURVEY

Efficient LLMs Training and Inference: An Introduction

RUI LI^{ID}, DEJI FU, CHUNYU SHI, ZHILAN HUANG, AND GANG LU^{ID}

Cloud and Network Operation Technology Department, China Telecom Research Institute, Guangzhou 100033, China

Corresponding author: Rui Li (lir33@chinatelecom.cn)

ABSTRACT ChatGPT was released in late November 2022, making a significant impact globally. Following this release, numerous domestic and international open-source projects for large model training emerged, including Alpaca, BOOLM, LLaMA, ChatGLM, DeepSpeedChat, and ColossalChat. Both academia and industry have a growing need to train large models for optimizing downstream tasks. Research has demonstrated that instruction tuning using high-quality training data can enable models with over 10 billion parameters to exhibit emergent abilities, particularly in complex reasoning tasks. Training a model with tens of billions of parameters is often necessary to improve business metrics using large model technology. However, most research teams lack the extensive computing resources required for such large-scale models, often possessing only a few V100 (32G) GPUs and occasionally a few A100s. Consequently, training or inferring a large model of this magnitude is impractical under limited computational resources. Therefore, adopting optimization strategies during the training and inference stages is essential to address these challenges. This survey summarizes a series of optimization techniques for large models during the training and inference phases, enabling the training of large models even with constrained computational resources.

INDEX TERMS Large language models, training, inference, optimization.

I. INTRODUCTION

Since its release at the end of 2022, ChatGPT has rapidly gained widespread attention, permeating social media platforms and major websites, and becoming a dominant topic of discussion [1]. The model's rapid and widespread adoption has been remarkable. The fervor surrounding ChatGPT has ignited a global interest in AI, revitalizing the field. ChatGPT's achievements exemplify the significant advancements in general artificial intelligence, marking a leap forward in domains such as automated programming [2], scientific discovery [3], personalized digital assistants [4], creative arts [5], and next-generation computing architecture [6], thereby demonstrating AI's versatility and substantial impact across industries.

However, the remarkable success of large language models (LLMs) has introduced several challenges, particularly their substantial computational demands during training and deployment. The vast model size and complexity, along with

the requirement for extensive computational resources, have hindered their widespread use in real-world applications. The resource-intensive nature of these models raises concerns about energy consumption, scalability, and accessibility, limiting their adoption by broader communities without significant computational resources, such as large companies.

This introduction paper aims to address the critical need for efficient LLM training and serving by thoroughly exploring the diverse strategies proposed by the research community to tackle this challenge. We provide an in-depth examination of the full range of solutions, from architecture evolution to training and inferencing optimization techniques, all designed to optimize the performance of large language models.

The remainder of this paper is structured as follows: Section II introduces the background information about large language models. Section III introduces the architecture evolution roadmap and computation overhead of LLMs during the training and inference processes. Section IV discusses optimization techniques for training LLMs. Afterward, we introduce some representative LLM serving optimization

The associate editor coordinating the review of this manuscript and approving it for publication was Ramoni Adeogun^{ID}.

techniques in Section V. Finally, we propose some promising directions in Section VI for improving generative LLM training and serving efficiency to motivate future research.

II. BACKGROUND

AI large models, commonly known as ‘pre-trained large language models,’ are regarded as the cornerstone of modern machine learning. These models encapsulate two essential elements: ‘pre-training’ and ‘large models.’ The integration of these elements has led to the emergence of a new paradigm in artificial intelligence. These models undergo rigorous pre-training on large-scale datasets, thereby eliminating the need for extensive fine-tuning processes. With minimal data adjustments, they can be efficiently adapted to various domains. These models function like a versatile tool, capable of addressing a wide range of tasks. For example, a pre-trained language model can adeptly handle tasks such as natural language classification, named entity recognition, and coreference resolution, as well as manage natural language generation tasks like intelligent dialogue, reading comprehension, and text generation.

A. TRANSFORMER MODEL ARCHITECTURE

Transformer model origin: the foundation of ChatGPT’s underlying framework is derived from the Transformer model released by Google in 2017. This model introduced the attention mechanism, completely revolutionizing the traditional deep learning structures of recurrent and convolutional networks, showcasing the ambition to achieve a “unified model.” The Transformer architecture originated from a highly acclaimed paper [7].

Transformer is a novel end-to-end model that cleverly captures contextual information using an attention mechanism. Its overall structure consists of an encoder and a decoder, offering great flexibility. This model is widely used in various tasks, including translating one language into another and converting text from images into readable text in Optical Character Recognition (OCR) tasks. Hence, this model is aptly named “Transformer.”

1) END-TO-END MODEL COMPRISING ENCODER AND DECODER

The transformer model is a masterpiece that seamlessly integrates the encoder-decoder concept with the attention mechanism. Structurally, the GPT model only utilizes the decoder portion of the Transformer model, but the purposes of GPT and Transformer are vastly different. The Transformer was created to address practical tasks such as translation, while GPT emerged from the concept of advanced pre-training.

2) LARGE LANGUAGE MODEL (LLM) ARCHITECTURE: ENCODER AND DECODER

Currently, many large language models (LLMs) adopt a decoder-only architecture. In my observation, OpenAI has provided a successful example for the entire industry, leading

more people to follow their approach. However, before GPT-3, the encoder-only architecture represented by BERT was mainstream. With OpenAI’s continuous efforts, including the creation of large models with tens of billions of parameters, extensive pre-training, and the use of high-quality corpus data, the advantages of decoder-only models have gradually become apparent, especially in generative tasks.

In the future, we can expect to see competition and exploration between encoder-decoder and decoder-only architectures, which will drive further development and improvement of model architectures. In different application scenarios and tasks, different architectures may have their own advantages, so the choice of model may depend on specific needs and performance requirements.

3) THE ATTENTION MECHANISM

The attention mechanism is a method used for processing textual data, which addresses the limitations of recurrent neural networks (RNNs) in truly capturing bidirectional context and parallel computation through convolution calculations. To simulate the human characteristic of having different levels of focus on various things, three vectors, and corresponding calculation methods were proposed to emulate this behavior. For instance, the $\mathbf{Q}$ vector can be seen as a person’s focus, the $\mathbf{V}$ vector as the specific objects, and the $\mathbf{K}$ vector as the person’s attention level to different objects. By computing the dot product of the $\mathbf{Q}$ vector and the $\mathbf{K}$ vector, a value can be obtained that represents a person’s level of attention to a particular object. This attention level is then multiplied by the $\mathbf{V}$ vector to represent how the object appears in the person’s view. This method enables the model to better capture the relevance and importance of different parts of the text.

B. THE KEY TECHNICAL PRINCIPLE: PRE-TRAINED MODELS

The principle of pre-trained large models is based on the concepts of deep learning and self-supervised learning. Its core idea is to train a general neural network model through large-scale unsupervised learning and then fine-tune it on specific tasks to adapt it to particular applications. This approach has achieved tremendous success in fields such as natural language processing and computer vision, enabling models to perform excellently on various tasks.

Pre-training consists of two steps. The first step is pre-training, where a task-agnostic model is obtained from large-scale data through self-supervised learning. This step helps the model understand the semantic representation of words in different contexts. The second step is fine-tuning, where the model is adjusted for specific tasks. The training data can be text, text-image pairs, or text-video pairs. The training methods for pre-training models can use self-supervised learning techniques, such as autoregressive language models and autoencoder techniques. These models can be monolingual, multilingual, or multimodal. After fine-tuning, these models can be used for various tasks, including

classification, sequence labeling, structure prediction, and sequence generation. They can also be applied to build applications such as summarization, machine translation, image retrieval, and video annotation.

The development trends of pre-trained models include: first, the scale of the models is increasing, with the number of parameters growing significantly, such as from GPT-2's 150 million parameters to GPT-3's astonishing 175 billion parameters. Second, pre-training methods are continuously expanding, from autoregressive language models to various autoencoding techniques, as well as multi-task training. Third, the application domains of the models are constantly expanding, covering areas such as language, multilingual, code, and multimodal applications. Finally, research is also ongoing in model compression to enable these models to run efficiently in resource-constrained environments like mobile devices.

III. EVOLUTION AND PERFORMANCE ANALYSIS OF LARGE MODELS

In this section, we first introduce the variants of Large Models and major improvement methods of LLMs. After that, we analyze the computation demand of the training and inference process of LLMs.

A. THE VARIANTS OF LARGE MODELS

Variants of large models refer to the different adaptations and modifications made to the foundational architectures, such as the Transformer model, to enhance their performance. These modifications often aim to address specific limitations, improve computational efficiency, or better suit particular tasks.

Key improvements include:

- 1) **Structural adjustments:** Modifying the architecture of models like Transformers by introducing new components or altering existing ones, such as variations in attention mechanisms [8] or feed-forward networks [9].
- 2) **Efficiency enhancements:** Optimizations like reducing computational complexity or memory usage through techniques like sparse attention [10] or efficient encoding [11].
- 3) **Task-specific tuning:** Adapting the model's structure or pretraining objectives to perform better on specific tasks, such as language understanding or translation.

These changes are typically driven by the need for better accuracy, faster processing, or lower resource consumption in various applications of large models.

The rapid evolution of attention-based large models initially involved many variations in model architecture. Recently, these architectures have begun to converge towards a Decoder-only design, primarily focusing on improving and optimizing component modules. Many domestically developed models also modify the component modules of the original models.

The paper [12] explores the effectiveness of different language model architectures, pretraining objectives, adaptation

strategies, and finetuning methods in zero-shot generalization tasks. The authors conducted a comprehensive study using six pre-trained models, each with over 5 billion parameters, trained on 168 billion tokens. The models were evaluated on 30 tasks across two different benchmarks.

Key findings include:

- 1) **Architecture:** The study contrasts the performance of Causal Decoder (CD) [13], Non-Causal Decoder (ND) [14], and Encoder-Decoder (ED) models [15]. It finds that ED models generally outperform CD models in transfer learning scenarios, especially when multitask finetuning is involved.
- 2) **Pretraining objective:** The study compares different pretraining objectives such as Full Language Modeling (FLM) [16], Prefix LM (PLM), and Masked LM (MLM) [17]. It suggests that the choice of pretraining objective significantly impacts the model's zero-shot generalization capabilities.
- 3) **Adaptation and finetuning:** The study highlights the importance of adaptation (e.g., MLM adaptation) and multitask finetuning. It finds that models with these additional steps tend to show improved zero-shot generalization.

Overall, the paper [12] concludes that multitask fine-tuned ED models with specific adaptations are particularly effective for zero-shot generalization, making them a strong choice for tasks requiring minimal training data.

B. THE TAXONOMY OF LARGE MODELS

A wide variety of models have been proposed so far based on the vanilla Transformer from three perspectives: types of architecture modification, pre-training methods, and applications [18].

Primarily categorize the existing variants from the following three perspectives:

- 1) **Architectural modification:** This category involves direct modifications to the original Transformer model architecture to improve efficiency, enhance generalization, or adapt to specific applications. Specific modifications include:
 - a) **Module-level modifications:** Improvements to components such as the attention mechanism [19], [20], positional encoding [21], [22], layer normalization [23], [24], and feedforward network [25], [26].
 - b) **Architecture-level variants:** Examples include lightweight Transformers [27], improved inter-block connections [28], adaptive computation time [29], divide-and-conquer strategies [30], and alternative architectures [31].
- 2) **Pre-training methods:** This category involves pre-training Transformer models on large corpora to learn general language representations that are beneficial for downstream tasks. Pre-training methods include:
 - a) **Encoder models:** Examples like BERT and RoBERTa use masked language modeling (MLM) [32] and next

sentence prediction (NSP) as self-supervised training objectives [33].

- b) **Decoder models:** The GPT series focuses on pre-training Transformer decoders for language modeling [34].
- c) **Encoder-decoder models:** Models like BART [35] and T5 [36] combine encoder and decoder architectures for both understanding and generating natural language [37].
- 3) **Applications:** This category involves applying Transformer models to various fields, including but not limited to:
 - a) **Natural language processing (NLP):** Examples include machine translation, language modeling, and named entity recognition [38].
 - b) **Computer vision (CV):** Applications such as image classification, object detection, and video processing [39].
 - c) **Audio processing:** Including speech recognition, speech synthesis, and music generation [40].
 - d) **Multimodal applications:** Such as visual question answering, visual commonsense reasoning, image captioning, speech-to-text translation, and text-to-image generation [41].

C. THE COMPUTATION AND MEMORY DEMANDS OF LLMs TRAINING

1) COMPUTATION DEMAND

There is no fixed formula for calculating the computational power required for training large models because it is influenced by various factors, including model size, the amount of training data, training time, optimization algorithms, and hardware performance. However, we have a rule of thumb to estimate the computational demand of the training process of LLMs [42]:

$$C \approx 6ND, \quad (1)$$

where C is the computational demand, N is the size of the model parameter, D is the size of the training data (the number of the training tokens), and 6 is the coefficient that describes the computational demand FLOPs per model parameter per training token. For example, training a 34B model with 5B tokens corpus volume needs 1.02×10^{22} FLOPs.

2) MEMORY DEMAND

Memory demands during training are substantial, with GPU memory being a critical resource. LLMs can require hundreds of gigabytes of memory to store model parameters, activations, and intermediate results. The total memory demand of the training process of LLMs can be summarized as

$$\begin{aligned} \text{TotalMemory} = & \text{ModelSize} + \text{KVcache} + \text{Activations} \\ & + (\text{OptimizerStates} + \text{Gradients}) \\ & * \text{NumberofTrainableParameters} \end{aligned} \quad (2)$$

- 1) **Model weight:** The first and most important memory requirement is the memory needed for loading the model. This depends on the number of parameters of the model and the precision:

$$\text{ModelSize} = \text{NumberofParameters} * \text{Precision} \quad (3)$$

- 2) **KV cache:** In transformers, the decoding phase generates a single token at each time step, dependent on previous token tensors. To avoid recomputing these tensors, they are cached in the GPU memory:

$$\begin{aligned} \text{KVCache} = & 2 * \text{BatchSize} * \text{SequenceLength} \\ & * \text{NumberofLayers} * \text{HiddenSize} \\ & * \text{Precision} \end{aligned} \quad (4)$$

- 3) **Activation:** Intermediate activation values must be stored during the forward pass of the model. These activations represent the outputs of each layer in the neural network as data propagates forward through the model. They must be kept in FP32 to avoid numerical explosion and ensure convergence:

$$\begin{aligned} \text{ActivationMemory} \\ = & \text{BatchSize} * \text{SequenceLength} \\ & * \text{HiddenSize} * (34 + (5 * \text{SequenceLength} \\ & * \text{Numberofattentionheads}) / (\text{HiddenSize})) \end{aligned} \quad (5)$$

- 4) **Optimizer states:** Optimization algorithms require resources to store the parameters and auxiliary variables. These variables include parameters like the momentum and variance used by optimization algorithms such as Adam (2 states) or SGD (1 state). This depends on the number of optimized states and their precision.
- 5) **Gradients:** Gradient values are computed during the backward pass of the model. They represent the rate of change of the loss function concerning each model parameter and are crucial for updating the parameters during optimization. As activations, they must be stored in FP32 to maintain numerical stability.

D. THE COMPUTATION AND MEMORY DEMANDS OF LLMs INFERENCE

1) UNDERSTANDING LLM INFERENCE

Many widely-used decoder-only LLMs, such as GPT-3, are pretrained using a causal modeling objective, functioning primarily as next-word predictors. These models receive a sequence of tokens and generate the following tokens one by one, continuing this process until they either reach a predefined stopping point (like a maximum token count or a stop word list) or produce a special <end> token signaling the completion of the generation. This generation process is divided into two main stages: the prefill phase and the decode phase.

- 1) **Prefill phase or processing the input:** During the prefill phase, the LLM processes the input tokens to calculate the intermediate states (keys and values)

needed to produce the initial new token. While each subsequent token relies on all preceding tokens, the complete input is known in advance, making this process a highly parallelized matrix-matrix operation. This efficiently maximizes GPU utilization.

- 2) **Decode phase or generating the output:** During the decode phase, the LLM produces output tokens one by one in an autoregressive manner until a stopping condition is reached. Each new token relies on the output states (keys and values) from previous iterations. This phase functions similarly to a matrix-vector operation, which doesn't fully utilize GPU computing power compared to the prefill phase. The latency is primarily determined by the speed at which data (weights, keys, values, activations) is transferred from memory to the GPU, rather than the speed of the computation itself. Thus, this phase is considered memory-bound.

2) COMPUTATION DEMAND

The computational requirements for inference include the number of computations and memory accesses, which can be predicted through mathematical modeling and used to verify the performance of the inference system. As the complexity of the model increases, the demands on computation and memory will significantly rise, necessitating multi-card parallel processing to meet the requirements of the inference task. We have a rule of thumb to estimate the computational demand of the inference process of LLMs:

$$C \approx 2NS, \quad (6)$$

where C is the computational demand, N is the size of the model parameter, S is the sequence length of the input and output token, and 2 is the coefficient that describes the computational demand FLOPs per model parameter per token.

3) MEMORY DEMAND

Performing inference requires resources for loading the model weights and storing the KV cache and activation memory. It doesn't require gradients and optimizer states as these are only required for training:

$$\text{TotalInferenceMemory} = \text{ModelSize} + \text{KVCache} + \text{Activations}. \quad (7)$$

This section introduces the variants of large models and major improvement methods of LLMs, along with the taxonomy of large models. After that, we present the computation and memory demands of LLMs training and inference processes. As we can see, training or inferring a large model of such magnitude computation and memory demand is impractical under limited resources. In the next two sections, we will introduce a series of optimization techniques for large models during the training and inference phases, enabling the training of large models even with constrained computational resources.

IV. THE OPTIMIZATION TECHNIQUES OF TRAINING LLMs

A. THE CHALLENGES OF TRAINING LLMs

In recent years, the parameter size of large models has been increasing at a rate of 10 times per year. Before the era of large models, training an AI model could usually be handled by a single machine with a single GPU, or at most, multiple GPUs, with training periods ranging from hours to a few days. However, training a large model with a trillion parameters now requires a large distributed cluster involving hundreds of servers and thousands of GPUs or XPU cards, extending the training period to months.

For instance, training GPT-3 with 175 billion parameters and 300 billion tokens would take one A100 GPU approximately 32 years, based on peak performance at half-precision, while 1,024 A100 GPUs would take 34 days at 45% resource utilization. Moreover, even if time was not an issue, a single A100 GPU would be unable to train a model with a trillion parameters due to its insufficient memory capacity.

When training large models in a distributed environment, the training period is reduced from decades on a single card to just days. However, this requires overcoming challenges such as computational limits, memory limits, and communication bottlenecks to fully utilize all resources within the cluster and speed up the training process.

The computational limit, or "compute wall," refers to the vast difference between the computing power of a single GPU and the total computing power required by the model. For example, the A100 GPU has a compute power of only 312 TFLOPS, while GPT-3 requires 314 ZFLOPs, a difference of nine orders of magnitude.

The memory limit, or "memory wall," refers to the inability of a single GPU to store all the parameters of a large model. For instance, GPT-3's 175 billion parameters require 700 GB of memory (assuming 4 bytes per parameter), while the NVIDIA A100 GPU only has 80 GB of memory.

The essence of both the compute wall and memory wall is the contradiction between the limited capacity of a single GPU and the enormous storage and computational demands of the model. This can be solved through distributed training, but it introduces a new challenge: the communication bottleneck or "communication wall."

The communication wall arises when distributed training requires frequent synchronization of parameters among the cluster's compute units, which can slow down the overall computation speed. If not handled properly, this could mean that increasing the cluster size actually reduces training efficiency. Successfully overcoming the communication wall is indicated by the strong scalability of the cluster, where the acceleration from multiple GPUs scales with the size of the cluster. The linear acceleration ratio is the metric used to evaluate this capability, with higher values being better.

These walls begin to appear in multi-machine, multi-GPU training. As the parameters of large models continue to grow, so do the corresponding cluster sizes and the height of these walls. Additionally, during long-term training in large

TABLE 1. The widely used training frameworks.

	computation optimization			memory optimization			parallel optimization				sparsification	
	operator optimization (FlashAt)	operator selection (AdamW/SGD)	mixed precision training (AMP/apex)	memory offloading (Zero1/2/3/Infinity)	gradient check pointing	gradient accumulation	pipeline parallelism (PP)	data parallelism (DP)	tensor parallelism (TP)	expert parallelism (EP)	model sparsity (MoE)	operator sparsity (SparseAtt)
Microsoft Megatron-DeepSpeed [43]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Nvidia MegaTron-LM [44]	✓	✓	✓		✓	✓	✓	✓	✓		✓	✓
Huggingface Accelerate [45]		✓	✓		✓	✓	✓	✓	✓		✓	
Colossal-AI [46]		✓	✓	✓	✓	✓	✓	✓	✓		✓	
Meta FairScale [47]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Baidu PaddlePaddle [48]		✓	✓	✓	✓	✓	✓	✓	✓		✓	✓
BAAI FlagScale		✓	✓		✓	✓	✓	✓	✓		✓	

clusters, hardware failures may occur, potentially affecting or interrupting the training process.

B. OPTIMIZATION TECHNIQUES

Training large models requires comprehensive optimization across dimensions such as data loading, model computation, parallel communication, and memory optimization. Fig. 1 shows how these techniques influence the final training throughput and some representative examples of these techniques.

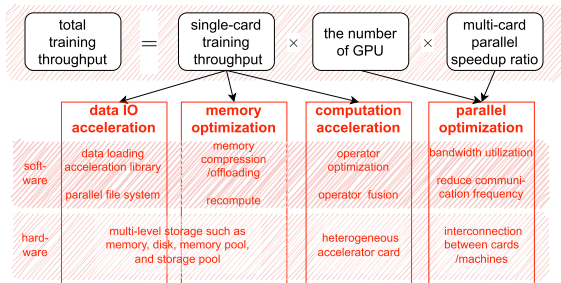


FIGURE 1. Representative optimization examples of data loading, model computation, parallel communication, and memory optimization.

Fig. 1 shows the main four optimization techniques of LLMs training. As we can see, the total training throughput is influenced by the single-card training throughput, the number of GPU cards, and the multi-card parallel speedup ratio.

C. LARGE MODEL TRAINING FRAMEWORK

Large model training frameworks are critical tools in modern AI research, designed to support the efficient training of large-scale language models such as the GPT series and Llama. These frameworks typically enhance training efficiency and reduce computational costs through various parallelization techniques and optimization algorithms. Prominent international training frameworks include DeepSpeed [49] and Megatron-LM [44], each with its unique advantages. Table 1 lists the widely used training frameworks and their assembled optimization techniques:

1) **Megatron-DeepSpeed**: is a framework jointly developed by NVIDIA and Microsoft, combining

Megatron-LM's large-scale model training capabilities with DeepSpeed's efficient distributed training technology. Its advantage lies in the ability to significantly reduce memory usage through ZeRO optimization, making the training of ultra-large-scale models more efficient, while fully leveraging GPU clusters for large-scale parallel computing, making it suitable for training large language models with billions of parameters.

- 2) **Megatron-LM**: developed by NVIDIA, is a distributed training framework based on PyTorch, specifically designed for training large-scale Transformer models. Its key features include data parallelism, tensor parallelism, and pipeline parallelism, which together significantly enhance the efficiency of training ultra-large models. The main advantage of Megatron-LM lies in its ability to greatly reduce training time for large models while fully utilizing GPU clusters, making it ideal for training models with billions of parameters. Additionally, it features innovative designs in model parallelization.
- 3) **Huggingface accelerate**: is a powerful library that enables the same PyTorch code to run across any distributed configuration by adding just four lines of code. Its advantages lie in simplifying the complex process of distributed training, allowing developers to deploy large-scale Natural Language Processing (NLP) models more quickly and efficiently across various environments. This tool is particularly suited for applications requiring high-performance computing
- 4) **PaddlePaddle**: is an open-source deep learning framework developed by Baidu, offering extensive functionality and wide application scenarios. Its advantages include comprehensive support for model development in areas such as Natural Language Processing (NLP), Computer Vision (CV), and recommendation systems. It also provides efficient parallel computing capabilities and user-friendly APIs. Additionally, PaddlePaddle is backed by a strong community, making it easy to use, especially for industrial-grade applications.

TABLE 2. The functions of the four optimization techniques.

	computation optimization			memory optimization			parallel optimization				sparsification	
	operator optimization	operator selection	mixed precision training	memory offloading	gradient checkpointing	gradient accumulation	pipeline parallelism	data parallelism	tensor parallelism	expert parallelism	model sparsity	operator sparsity
reducing computation demand	✓	✓	✓				✓	✓	✓	✓	✓	✓
reducing memory usage		✓	✓	✓	✓	✓					✓	✓
reducing communication demand	✓	✓	✓								✓	

AI frameworks do not support all optimization techniques simultaneously. Megatron-DeepSpeed offers the most comprehensive set of features, Accelerate is the easiest to use, Colossal-AI saves the most GPU memory, and PaddlePaddle supports a larger number of domestic models. The mainstream frameworks are still Megatron-DeepSpeed, with some domestic frameworks based on Megatron for further development. Engineering translation of cutting-edge research results still experiences delays, typically taking about 6 months to 1 year. Domestic frameworks generally have mainstream optimization methods, but the adoption of advanced optimization techniques is slower. Some memory optimization techniques may slow down training speed but are suitable for scenarios with limited hardware resources.

Computation, memory, and communication are inter-dependent constraints, so optimization techniques need to be arranged and combined according to the specific scenario. Saving memory, communication overhead, and computational acceleration are conflicting factors. To achieve the highest training speed and throughput, it's necessary to enhance training speed and throughput while reducing communication and memory consumption on limited hardware. This requires seeking the optimal combination of optimizations. Here are the comments on the four optimization techniques:

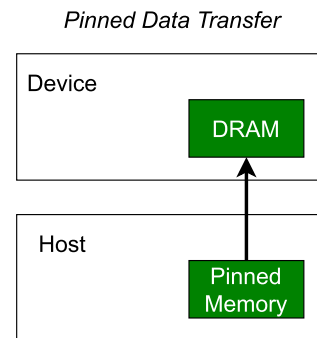
- 1) **Computational optimization** primarily focuses on reducing the amount of computation and minimizing memory access within a single GPU card, typically without increasing communication overhead.
- 2) **Memory optimization** often trades off time for space, which increases computation and communication overhead, thereby delaying training duration, but it can save hardware resources.
- 3) **Parallel optimization** involves various combinations of parallelism. If there is no high-speed interconnect between GPUs, it is recommended not to use tensor parallelism (TP). This is especially true if both the interconnect speed and GPU memory are relatively small.
- 4) **Sparsification** is a research hotspot that typically yields significant benefits but can also result in some accuracy loss and increased communication overhead.

D. DATA IO OPTIMIZATION

During the training of large models, the data IO faces the following major challenges:

- 1) **Reading and processing massive amounts of data:** Large models typically need to handle petabyte-scale data. Efficiently reading and preprocessing this data becomes a key challenge in the training process.
- 2) **Storage capacity and bandwidth limitations:** As the data volume and model complexity increase, the capacity and IO bandwidth of storage devices become bottlenecks. Traditional storage systems may not meet the high concurrent data read requirements of large models, leading to IO operation delays.
- 3) **Mismatch between IO bottlenecks and computational resources:** During the training of large models, IO performance often lags behind the demands of computational resources. Even with powerful GPUs or TPUs, the speed of IO operations can limit training efficiency.
- 4) **Complexity of data management:** Data management, distribution, and caching become extremely complex, especially in distributed systems. Ensuring efficient data transmission and sharing across multiple computing nodes is another major challenge.

Addressing these IO challenges is crucial for improving the efficiency of large model training and typically requires a combination of optimized data preprocessing, parallel IO operations, and distributed storage systems. The following shows two major optimization techniques for solving IO challenges:

**FIGURE 2.** The illustration of pinned memory.

1) PINNED MEMORY

Pinned Memory [50] (also known as Page Locked Memory) is a special type of memory allocation: preloading the data tensors into pinned memory instead of regular memory allows

them to be directly accessed by the GPU without needing to pass through the CPU. The benefit of this approach is that it reduces the number of data transfers between the CPU and GPU, thereby lowering the overhead of data transfer and speeding up the training process.

Pinned memory refers to a region of memory allocated in the host (CPU) that is locked by the operating system to prevent it from being moved into physical memory or swapped to virtual memory (i.e., disk space). The primary purpose of this is to improve the efficiency of data transfer between the host and devices such as GPUs. The main characteristics are as follows:

- 1) **Fixed location:** Pinned memory is fixed at a specific location in physical memory and will not be moved to virtual memory by the operating system due to memory pressure. This ensures the stability of memory addresses, which is crucial for hardware peripherals that require direct memory access (DMA), such as GPUs.
- 2) **Increased transfer efficiency:** Since pinned memory does not move, the DMA controller on the GPU can directly access these memory regions without involving the CPU. This reduces the CPU's workload and increases the speed and efficiency of data transfer.
- 3) **Allocation and release:** In CUDA, pinned memory can be allocated through the CUDA driver API or runtime API. When releasing it, the corresponding function should be used. Additionally, memory can be allocated on the host using the Malloc() function and then registered as pinned memory using the cudaHostRegister() function.

2) KSPEED

As shown in Fig. 3, data is first read from the disk into the host cache and then transferred from the host cache to the accelerator card's HBM memory via PCIe. In a distributed training system, the bandwidth of the data IO process is limited by the speed of PCIe. If the IO rate cannot keep up with the rate at which the accelerator card consumes data during computation, data IO will become the bottleneck of the entire training system.

To minimize the overhead of IO read operations, the Alibaba development team created the KSpeed IO

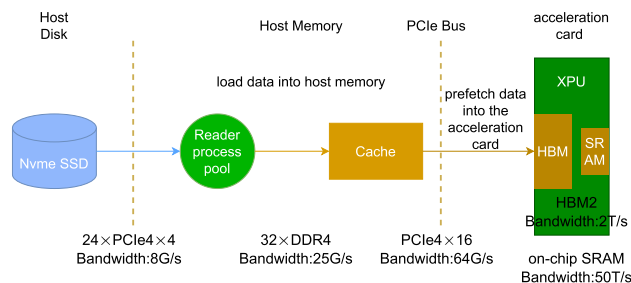


FIGURE 3. The illustration of how data be transferred from disk to on-chip SRAM.

library [51]. This library not only enables near-end caching in a compute-storage separation architecture, placing data as close as possible to where it is consumed but also reconstructs the entire AI data pipeline by fully parallelizing data preprocessing and loading, resulting in up to a 10x improvement in IO performance.

As shown in Fig. 4, the traditional train sequence pipelining generates iteration gaps (batch data loading phases) between two batch training processes. The IO access pattern of AI training data sets exhibits two key characteristics:

- 1) Each iteration involves known data sets accessed in a specific order.
- 2) Any two batches of data can be read in parallel without any dependencies.

After applying the KSpeed IO library, the training data IO is completely hidden from the computation process, except for the batch1 data loading.

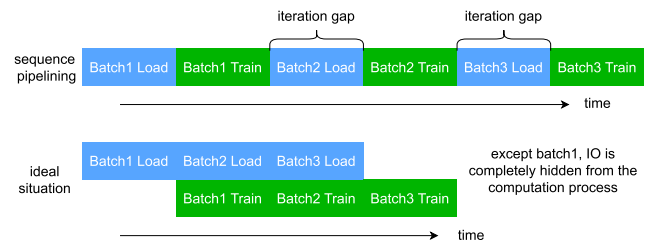


FIGURE 4. The illustration of batch data loading and training.

E. COMPUTATION OPTIMIZATION

1) OPERATOR FUSION

Operator fusion [52] is a key computation optimization technique used in large model training to improve the performance of deep learning models. The principle and impact of operator fusion can be summarized as follows:

Principle: Operator fusion works by combining multiple operations (or “operators”) into a single, more complex operation during the execution of a computational graph. In deep learning frameworks, models are typically represented as a graph of operations that process data through various layers. Each node in this graph corresponds to a mathematical operation like addition, multiplication, convolution, etc.

Instead of executing each operation separately, which involves multiple memory accesses and data transfers, operator fusion merges these operations into a single kernel. This fused kernel performs multiple operations in one pass, reducing the number of times data needs to be moved between different levels of memory (such as between GPU memory and registers) or between different computational units.

Therefore, operator fusion can bring the following benefits:

- 1) **Reduced memory bandwidth usage:** By minimizing the number of memory accesses, operator fusion significantly reduces the memory bandwidth requirement, which is often a bottleneck in deep learning workloads.

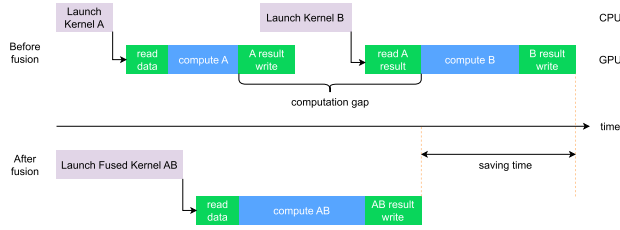


FIGURE 5. The illustration of time-saving of operator fusion.

- 2) **Lower latency:** Merging operations reduces the overhead associated with launching multiple kernels. This decreases the total latency of executing a sequence of operations.
- 3) **Improved computational efficiency:** Fused operations can better utilize the computational resources (like GPU cores) by increasing the ratio of compute to memory operations, leading to better overall hardware utilization.
- 4) **Energy efficiency:** Fewer memory accesses and more efficient computation lead to reduced power consumption, which is crucial for scaling large models in data centers.

Overall, operator fusion plays a crucial role in optimizing the performance of large-scale deep learning models, enabling faster training and inference while making more efficient use of hardware resources.

2) HIGH PERFORMANCE OPERATOR

High-performance operators (also known as custom and specialized kernels) refer to highly optimized, purpose-built code routines designed to perform specific operations more efficiently on particular hardware architectures. In the context of large-scale model training and high-performance computing, these kernels are tailored to leverage the unique capabilities of the underlying hardware, such as GPUs, TPUs, or other accelerators.

- 1) **Custom kernels:** These are kernels specifically written to optimize a particular operation or a set of operations in a deep learning model. Unlike general-purpose kernels provided by frameworks (like cuBLAS, cuDNN, etc.), custom kernels are designed for specific tasks and can be fine-tuned to match the exact needs of an application. A custom kernel could be developed to optimize a unique convolution operation in a neural network, taking into account specific input sizes, stride patterns, or padding requirements.
- 2) **Specialized kernels:** These kernels are designed to exploit specific hardware features or instruction sets that are not utilized by general-purpose libraries. Specialized kernels might use features like tensor cores in NVIDIA GPUs, which are designed to accelerate matrix multiplication operations used in deep learning. NVIDIA's TensorRT provides specialized kernels that are optimized for inference on NVIDIA GPUs, using lower precision operations (like FP16) to accelerate computations while maintaining accuracy.

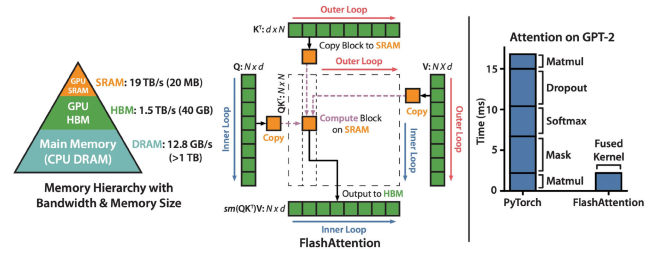


FIGURE 6. Source: <https://arxiv.org/pdf/2205.14135.pdf>. The illustration of flash attention.

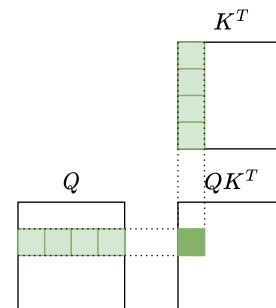
Flash Attention [53], [54] is a typical high-performance operator that is widely used in almost all LLM training frameworks. We know that the calculation formula for self-attention

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (8)$$

consists of several operations: matmul + div + masking + softmax + matmul. When the sequence length is large (e.g., 2048 or even up to 32K as in GPT-4), the Attention matrix will have a space complexity of $O(n^2)$. If these operations are computed on a single GPU, it will consume a significant amount of memory.

Flash Attention enables these Attention operations to be computed in a block-parallel manner. If these four operations can be processed in blocks, this goal can be achieved. The following introduces the block processing of each operation one by one:

- 1) QK^T : matrix multiplication operation can be parallelized using block matrices. As shown in the figure below, multiplying two matrices can be done by sliding block matrices along the rows of matrix Q and the columns of K^T (i.e., the rows of matrix K), and then summing the results of each block:



- 2) $\frac{QK^T}{\sqrt{d_k}}$: this step requires a division operation. Since division is an element-wise operation, it can be easily processed in blocks.
- 3) $\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$: this step is critical because it involves the softmax and multiplication operations. Specifically, softmax is neither a multiplication nor an element-wise operation; it normalizes each row of the matrix $\frac{QK^T}{\sqrt{d_k}}$. Therefore, a parallel processing strategy needs to be designed specifically for this operator. The team at

Stanford proposed the softmax Tiling strategy to achieve parallel and combined processing of the softmax and multiplication operations:

For example, let's say we need to compute the softmax of an array x . If we can only compute two numbers at a time, we first calculate the softmax of the first two numbers:

```
cur_sum = exp(x[0]) + exp(x[1])
y[0:2] = x[0:2] / cur_sum
pre_sum = cur_sum
```

Then, we calculate the softmax of the next two numbers, $x[2]$ and $x[3]$. At this point, cur_sum will be updated, and the previous sum is stored in the variable pre_sum . We can obtain the correct result by dividing the previous softmax results of the first two numbers by cur_sum/pre_sum . If the softmax is followed by a matmul, the result of the previous softmax will be multiplied by a block and accumulated. In this case, we only need to scale the accumulated value. By scaling the accumulated value in each iteration, we can incrementally compute the results of the softmax or the softmax + matmul.

3) PARAMETER-EFFICIENT LEARNING

Traditional model training involves updating all parameters during the learning process, which can be computationally expensive and memory-intensive. Parameter-efficient learning techniques aim to reduce the number of parameters that need to be updated or stored, leading to faster training, lower memory consumption, and more efficient inference. Key Techniques in Parameter-Efficient Learning:

- 1) **Low-rank factorization** decomposes weight matrices into products of lower-rank matrices. This reduces the number of parameters while retaining most of the model's expressiveness. Matrix factorization methods like Singular Value Decomposition (SVD) [55] can be used to approximate large matrices with lower-dimensional representations.
- 2) **Pruning** [56] involves removing less important weights or neurons from a model, leading to a sparse network. The sparsity reduces the number of parameters and can lead to faster retraining and inference. Magnitude-based pruning removes weights with small absolute values, under the assumption that these contribute less to the final output [57].
- 3) **Quantization** [58] reduces the precision of the model's parameters, typically from 32-bit floating-point to lower precision, such as 8-bit integers. This results in smaller models and faster computations. Post-training quantization converts a trained model's parameters to lower precision, enabling efficient deployment on resource-constrained devices.
- 4) **Knowledge distillation** [59] involves training a smaller, student model to mimic the behavior of a larger, teacher model. The student model learns to approximate the output of the teacher model, allowing it to achieve similar performance with fewer parameters. In natural language processing, distillation has been used to

compress large transformer models like BERT into smaller versions, such as DistilBERT.

- 5) **Parameter-efficient fine-tuning** [60]: This approach fine-tunes only a subset of the model's parameters while keeping the rest fixed. Methods like Adapter modules, which add small, task-specific layers to a pre-trained model, fall under this category. Fine-tuning a small set of task-specific parameters in a pre-trained transformer while freezing the bulk of the model can significantly reduce the number of parameters that need to be updated.
- 6) **Prompt-tuning and prefix-tuning**: In prompt-tuning [61], only the input prompt is modified while the model parameters remain fixed. Prefix-tuning [62] extends this idea by prepending trainable vectors (prefixes) to the input sequences, allowing the model to adapt to specific tasks with minimal parameter updates. Prefix-tuning has been shown to perform competitively with full fine-tuning in large language models like GPT while updating only a small fraction of the parameters.

As a typical example of parameter-efficient learning, low-rank adaptation (LoRA) has been widely adopted for tasks where the full model fine-tuning is too expensive or impractical, making it an effective approach for large-scale model adaptation. LoRA is a technique designed to efficiently fine-tune large language models by injecting trainable, low-rank decomposition matrices into the pre-trained model layers while keeping the original model weights frozen. This method significantly reduces the number of trainable parameters, leading to lower computational and memory costs without sacrificing much of the model's performance.

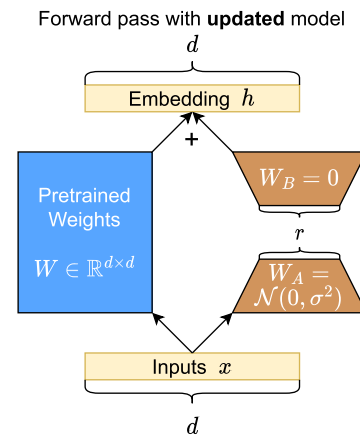


FIGURE 7. The illustration of LoRA.

As shown in the figure above, the blue section represents the original model parameters, which map the input x through a fully connected (FC) layer to h . However, the number of trainable parameters for the matrix $W \in \mathbb{R}^{d \times d}$ is $d \times d$. By adding a LoRA layer (orange section), the input x is first mapped to a lower-dimensional space and then mapped back to dimension d . At this point, the number of parameters required is only $2d \times r$, where r denotes the

rank of LoRA. During training, only the parameters in the orange section need to be trained and gradients need to be computed and stored, which significantly reduces the training overhead. However, introducing the LoRA parameters does not accelerate the inference phase, as the parameters in the red section still need to be involved in the forward computation. Therefore, the computational load during inference is expected to be slightly higher than the original.

F. MEMORY OPTIMIZATION

1) MIXED PRECISION TRAINING

Mixed precision training [63] is a commonly used memory optimization technique suitable for both single-GPU and multi-GPU parallel scenarios. Typically, floating-point operations in computers use FP32 (single precision), where 8 bits are allocated for the integer part and 23 bits for the fractional part, allowing for high-precision floating-point numbers.

In memory optimization scenarios, sacrificing some floating-point precision can reduce storage requirements. For example, using FP16 for floating-point operations requires only half the storage space, thus becoming half-precision floating-point operations. However, FP16 has a maximum integer value of 65,536, which can easily lead to overflow issues. To address this, BF16 is another half-precision floating-point representation that increases the storage bits for the integer part compared to FP16, mitigating overflow problems but sacrificing some precision.

In actual training, it is common to use a combination of single-precision and half-precision for floating-point operations. A typical example is dynamic mixed precision (automatic mixed precision, AMP) [64], as shown in the figure below:

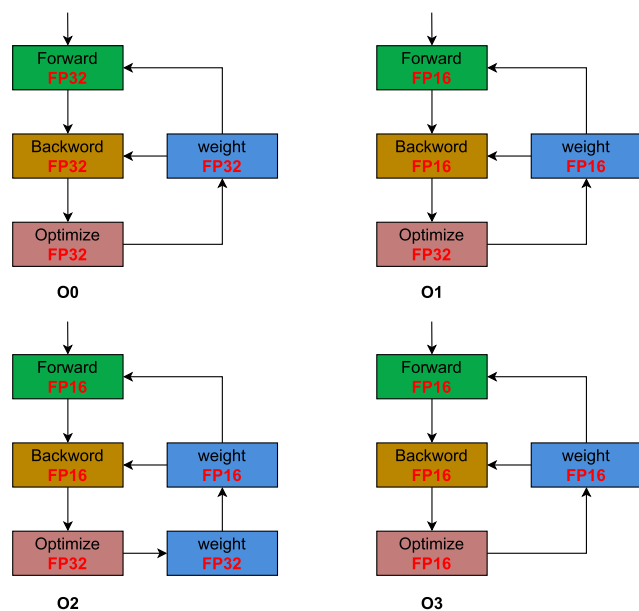


FIGURE 8. The illustration of mixed precision training.

- 1) **O0**: Represents the original FP32 floating-point operations.
- 2) **O1**: Uses FP32 for the optimizer part only, while the rest uses FP16.
- 3) **O2**: Builds upon O1 by additionally using FP32 to store a copy of parameters for updates.
- 4) **O3**: All parameters are in half precision.

AMP uses a mix of FP32 and FP16, dynamically specifying which parts to convert to half precision during different training stages. A typical approach of AMP is to use the O2 configuration shown above, where mixed precision training not only improves efficiency in multiplication operations but also effectively mitigates rounding errors during accumulation.

2) GRADIENT ACCUMULATION AND GRADIENT CHECKPOINTING

Gradient Accumulation [65] is a technique used in large model training, particularly useful when memory (e.g., GPU memory) is limited. It allows for the simulation of training with larger batch sizes by accumulating gradients over multiple smaller mini-batches and then updating the model parameters. The gradient accumulation mainly consists of three steps:

- 1) **Mini-batch calculation**: A large batch of data is split into several smaller mini-batches. Each mini-batch undergoes forward and backward propagation independently, computing the corresponding gradients.
- 2) **Gradient accumulation**: Instead of updating the model parameters immediately after each mini-batch, the gradients are accumulated across multiple mini-batches. Specifically, the gradients calculated after each mini-batch are added together.
- 3) **Parameter update**: Once the number of accumulated mini-batches reaches a predefined threshold (e.g., after 4 mini-batches), the model parameters are updated. At this point, the accumulated gradients are averaged by dividing by the number of mini-batches, thereby simulating the effect of training with a larger batch.

Assume a loss function L and model parameters θ , with a large batch size of B split into K mini-batches, each of size $b = B/K$. The mathematical formulas of gradient accumulation are:

- 1) For the k th mini-batch, the gradient is calculated as:

$$g_k = \frac{\partial L_k}{\partial \theta}$$

- 2) Gradient Accumulation:

$$G = \sum_{k=1}^K g_k$$

- 3) Parameter Update:

$$\theta = \theta - \eta \frac{G}{K}$$

where η is the learning rate.

By accumulating gradients over multiple mini-batches, gradient accumulation allows for the effective simulation of large-batch training under limited memory conditions. Gradient accumulation mimics large batch training, which can lead to more stable gradient estimates and benefit the stability and convergence speed of the training process.

While gradient accumulation saves memory, it may increase the overall training time since each large batch effectively involves multiple mini-batch computations. Moreover, when using gradient accumulation, it may be necessary to adjust the learning rate according to the size of the accumulated batches to avoid overly large or small gradient updates.

Gradient Checkpointing [66] is also a memory-saving technique used in the training of large models. It strategically trades off computational overhead to reduce memory usage, allowing the training of deep neural networks that would otherwise be too large to fit into the available memory.

Training deep neural networks requires a significant amount of memory to store activations (the outputs of each layer) during the forward pass, which are needed later during the backward pass to compute gradients. As models grow in size, the memory required to store these activations becomes a limiting factor, especially for models with many layers or large input sizes.

Gradient Checkpointing reduces memory usage by selectively storing activations (checkpoints) at a few layers during the forward pass instead of storing all activations. When it comes time to compute gradients during the backward pass, only the stored checkpoints are used. The activations for the other layers are recomputed on the fly during the backward pass. Here's a step-by-step breakdown of the process:

- 1) **Forward pass with checkpoints:** During the forward pass, instead of storing the activations of every layer, only the activations of selected layers (called checkpoints) are stored in memory. The layers between these checkpoints do not have their activations stored.
- 2) **Backward pass with recomputing:** During the backward pass, when gradients need to be computed, the activations between checkpoints are recomputed as needed. The recomputation starts from the nearest checkpoint and proceeds forward until the required layer's activation is recalculated.
- 3) **Gradient computation:** Once the necessary activations are available, the gradient for that layer is computed as usual.

By reducing the number of activations stored during the forward pass, gradient checkpointing can significantly decrease memory usage. This allows for the training of larger models or the use of larger batch sizes within the same memory constraints. However, the memory savings come at the cost of additional computation. Since some activations are recomputed during the backward pass, the overall computational workload increases. This leads to longer training times, as the recomputation adds overhead.

Gradient checkpointing is a powerful tool for managing memory during the training of large models, particularly when working with resource-constrained environments. It balances the need for memory efficiency with the increased computational demands, making it a valuable technique for training deep and complex neural networks.

3) ZERO SERIES

ZeRO (Zero Redundancy Optimizer) [67] is a set of techniques designed to optimize memory and computational efficiency in large-scale deep learning model training. ZeRO techniques are developed by Microsoft as part of the DeepSpeed library, and they aim to enable the training of very large models by reducing memory redundancy and optimizing resource utilization. These are the foundational stages of the ZeRO optimization approach:

- 1) **ZeRO-1 (Optimizer state partitioning):** ZeRO-1 focuses on partitioning the optimizer states (e.g., momentum, variance) across multiple GPUs. In traditional data-parallel training, each GPU stores a copy of the entire model's optimizer states, which leads to memory redundancy. ZeRO-1 eliminates this redundancy by distributing the optimizer states across GPUs. This allows larger models to be trained by freeing up memory that would otherwise be duplicated on every GPU.
- 2) **ZeRO-2 (Gradient partitioning):** ZeRO-2 extends ZeRO-1 by also partitioning the gradients across GPUs. Normally, after the backward pass, each GPU stores a complete copy of the gradients for the entire model. ZeRO-2 partitions these gradients, reducing memory usage further and enabling larger batch sizes or deeper models.
- 3) **ZeRO-3 (Parameter partitioning):** ZeRO-3 takes memory optimization to the next level by partitioning the model parameters themselves across GPUs. In addition to optimizer states and gradients, the model's parameters are distributed across GPUs. During the forward and backward passes, only the necessary parameters are loaded and used by each GPU, reducing the overall memory footprint. ZeRO-3 allows for the training of models with tens or even hundreds of billions of parameters on hardware that would otherwise be unable to handle such large models.

ZeRO-Infinity [68] is an extension of the ZeRO optimization framework that pushes the boundaries of model training even further:

- 1) **Memory dis-aggregation:** ZeRO-Infinity introduces memory dis-aggregation, allowing model parameters, optimizer states, and gradients to be offloaded to both CPU memory and NVMe (non-volatile memory express) storage.
- 2) **Unified memory:** This technique combines CPU, GPU, and NVMe memory into a unified memory pool, which the model can draw from as needed. This effectively removes memory constraints, enabling the training of models with trillions of parameters.

- 3) **Dynamic scaling:** ZeRO-Infinity dynamically scales memory usage based on the available hardware, optimizing the balance between computation speed and memory access latency. It achieves this by selectively keeping frequently accessed data on faster GPU memory while offloading less frequently used data to slower storage.

ZeRO++ [69] is a further refinement of the ZeRO approach, focusing on improving efficiency and usability:

- 1) **Optimized communication:** ZeRO++ introduces more efficient communication strategies to reduce the overhead associated with partitioning and synchronization across GPUs. This results in faster training times and better scaling efficiency, especially on large GPU clusters.
- 2) **Hybrid parallelism:** ZeRO++ supports a hybrid approach combining data parallelism, model parallelism, and ZeRO optimizations. This allows for flexible and efficient training of large models, catering to a wider range of hardware configurations.
- 3) **Hybrid parallelism:** ZeRO++ supports a hybrid approach combining data parallelism, model parallelism, and ZeRO optimizations. This allows for flexible and efficient training of large models, catering to a wider range of hardware configurations.
- 4) **Ease of use:** ZeRO++ simplifies the integration of ZeRO optimizations into existing training pipelines, making it more accessible to researchers and practitioners without needing extensive modifications to their training code.

The summary of differences and evolution:

- 1) ZeRO-1, ZeRO-2, and ZeRO-3: These stages progressively optimize memory usage by partitioning optimizer states, gradients, and parameters across GPUs. Each stage builds on the previous one, enabling increasingly larger models to be trained.
- 2) ZeRO-Infinity: Extends the ZeRO framework by introducing memory disaggregation and dynamic scaling, allowing models with trillions of parameters to be trained using a unified memory pool that includes CPU and NVMe storage.
- 3) ZeRO++: Focuses on improving efficiency, scalability, and ease of use by optimizing communication, supporting hybrid parallelism, and simplifying integration into existing training pipelines.

The evolution from ZeRO-1 to ZeRO++ reflects a continuous effort to push the limits of model size and training efficiency, making it possible to train increasingly large and complex models on a variety of hardware setups.

G. PARALLELISM OPTIMIZATION

3D Parallelism is a comprehensive strategy for training large-scale models that integrates three forms of parallelism: data parallelism, tensor parallelism, and pipeline parallelism. Each of these techniques addresses different aspects of

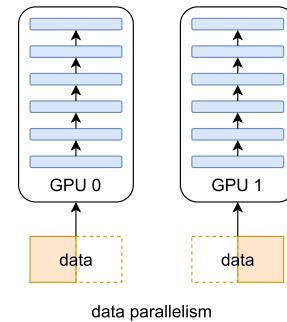


FIGURE 9. The illustration of data parallelism.

the computational and memory challenges in training large models, and they can be combined to optimize resource utilization and efficiency.

1) DATA PARALLELISM

Data parallelism [70] involves splitting the input data across multiple GPUs or nodes, with each processing a different subset of the data. The model parameters are replicated across all GPUs, and after each forward and backward pass, gradients are averaged (or reduced) across all GPUs to ensure that all model replicas are synchronized.

1) Applicability:

- a) Small to Medium-Sized Models: Suitable when the model can fit within the memory of a single GPU, but the dataset is large and needs to be processed in parallel to speed up training.
- b) Scaling Across Multiple GPUs: Effective for distributed training when multiple GPUs are available, and the model size is not the primary bottleneck.

2) Advantages:

- a) Simplicity: Easy to implement, as it only requires duplicating the model on each GPU and splitting the data.
- b) Scalability: Efficiently scales with the number of GPUs, making it ideal for training on large datasets.

3) Disadvantages:

- a) Memory Redundancy: Each GPU stores a complete copy of the model, which can lead to memory inefficiency, especially for very large models.
- b) Communication Overhead: The need to synchronize gradients across GPUs after every batch can introduce significant communication overhead as the number of GPUs increases.

2) TENSOR PARALLELISM

Tensor Parallelism [71] involves splitting the model's tensors (such as weight matrices) across multiple GPUs. Instead of each GPU storing the entire model, different GPUs store and compute different parts of the model's layers. The results from these partial computations are then communicated between GPUs to produce the full output.

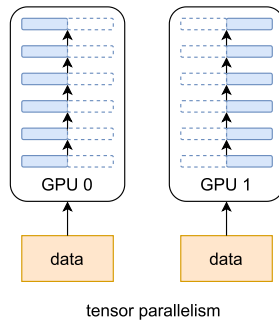


FIGURE 10. The illustration of tensor parallelism.

1) Applicability:

- a) Very Large Models: Tensor Parallelism is particularly useful when individual layers of the model are too large to fit on a single GPU.
- b) Deep Neural Networks: It is well-suited for deep neural networks with large layers (e.g., Transformer models), where each layer's operations can be split across GPUs.

2) Advantages:

- a) Memory Efficiency: By splitting tensors, the memory requirements per GPU are reduced, allowing for the training of much larger models.
- b) Scalability for Large Layers: Allows models with extremely large layers to be trained by distributing the computational burden across multiple GPUs.

3) Disadvantages:

- a) Complexity: Tensor splitting and the associated communication between GPUs add complexity to the implementation.
- b) Communication Overhead: The need to frequently exchange intermediate results between GPUs can introduce significant communication overhead, which may limit the speedup gained from parallelism.

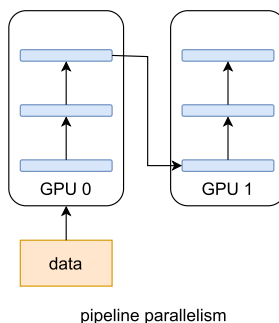


FIGURE 11. The illustration of pipeline of parallelism.

3) PIPELINE PARALLELISM

Pipeline Parallelism [72] divides the model into different stages, with each stage assigned to a different GPU. The input data is processed in a pipeline fashion: once the first

stage finishes processing one batch, it passes the intermediate results to the next stage, allowing multiple batches to be processed simultaneously in different stages of the pipeline.

1) Applicability:

- a) Deep and Wide Models: Useful for models with many layers where the computation can be divided into distinct stages.
- b) Memory-Constrained Environments: Ideal when a model is too large to fit into the memory of a single GPU, even after tensor partitioning.

2) Advantages:

- a) Memory Distribution: Reduces the memory burden on any single GPU by distributing the model across multiple GPUs.
- b) Improved Utilization: By overlapping the computation of different batches, it can lead to better utilization of GPU resources and reduce idle time.

3) Disadvantages:

- a) Pipeline Bubbles: The pipeline must be filled before it runs at full efficiency, leading to a delay in the start of full utilization (referred to as “pipeline bubbles”).
- b) Complexity in Implementation: Requires careful division of the model into stages and managing the inter-stage communication.

4) COMPARISON AND ANALYSIS

1) Data parallelism vs. tensor parallelism:

- a) Memory usage: Data parallelism requires each GPU to store the entire model, leading to redundancy. Tensor parallelism, on the other hand, distributes the model across GPUs, reducing memory usage.
- b) Communication: Data parallelism primarily involves gradient synchronization, whereas tensor parallelism involves frequent communication of intermediate results during forward and backward passes.

2) Data parallelism vs. pipeline parallelism:

- a) Applicability: Data parallelism is more straightforward and is typically used when the model can fit within a single GPU. Pipeline parallelism is used when the model is too large to fit on a single GPU and benefits from being divided into stages.
- b) Efficiency: Data parallelism can suffer from communication bottlenecks with large numbers of GPUs, while pipeline parallelism can face challenges like pipeline bubbles and requires more complex scheduling.

3) Tensor parallelism vs. pipeline parallelism:

- a) Layer granularity: Tensor parallelism works at the level of individual layers, splitting them across GPUs. Pipeline Parallelism works at a coarser granularity, splitting the entire model into stages.
- b) Communication overhead: Tensor parallelism has higher intra-layer communication overhead, while pipeline parallelism requires communication primarily between stages.

In practice, these three forms of parallelism can be combined to optimize training efficiency, especially for extremely large models: data parallelism(handles large datasets and scales across multiple GPUs), tensor parallelism(splits large layers within those GPUs), and pipeline parallelism(divides the model into stages to further distribute the computational load and memory requirements).

This 3D parallelism approach is highly flexible and can be tailored to fit the available hardware and the specific characteristics of the model being trained. For example, large-scale models like GPT-3 or other large Transformers often employ all three forms of parallelism to achieve efficient training on supercomputing clusters.

V. THE OPTIMIZATION TECHNIQUES OF SERVING LLMs

Recently, survey articles on LLM inference optimization have sprung up [73], [74], [75], [76]. These surveys provide a detailed overview of various efficient LLM inference techniques from multiple perspectives, including data level, model level, and system level. Many of these techniques are already widely used in common LLM inference frameworks, such as Continuous Batching [77], PagedAttention [78], FlashAttention [53], [54], FlashDecoding [79], and various quantization methods [80].

Despite the numerous optimization methods available, not all of them have been widely adopted. This is mainly because each optimization technique comes with its own costs, and the key is whether these costs can be outweighed by the benefits brought by the optimization. For example, using KV Cache can significantly save computational resources, but it also increases memory usage. Since the savings in computational resources far outweigh the increased memory cost, this method is highly cost-effective and widely adopted. Additionally, some optimization methods might sacrifice model accuracy, such as quantization and pruning techniques, so it is essential to carefully balance the trade-off between speed and accuracy. Finally, the generality and robustness of a technique are also crucial factors in determining its widespread adoption. If a technique requires separate optimization for each scenario and model, it will significantly increase the adaptation cost. Even if this optimization can bring some performance improvements, it might not be prioritized. Furthermore, some techniques, such as certain speculative sampling methods, only show advantages in small batch size scenarios, which limits their applicability. In this section, we only briefly introduce the widely adopted optimization techniques of LLM inference, interested readers can refer to the elaborate surveys [73], [74], [75], [76] for the wholistic introduction of LLM inference optimization techniques.

A. THE CHALLENGES OF INFERRING LLMs

The inference speed of LLMs is a crucial factor to consider in the production deployment of LLMs. There are mainly four metrics to measure LLM services, which are specifically introduced in this subsection. Before diving into these

metrics, a brief introduction to the two common modes of LLM inference is provided:

- 1) After a user inputs a query, the LLM model generates all outputs and returns them together. This approach is relatively simple to implement, but the problem is that LLM generation is usually slow, and waiting for the entire output to be generated and returned can result in long delays.
- 2) The streaming mode, on the other hand, generates tokens one by one during LLM inference. Since humans read gradually from left to right, it is natural to return the output as each token or a group of tokens is generated, allowing for some overlap between human reading and LLM generation. This can greatly improve the user experience (as seen in ChatGPT, which uses this method).

1) INFERENCE METRICS

- 1) **Time to first token (TTFT):** This metric refers to the time it takes for the model to generate the first output token after the user inputs a query. Since the tokenizer's speed in converting text to tokens and vice versa is very fast, it can be practically ignored, meaning this time is approximately equal to the prefill time. TTFT corresponds to the time at which the user begins to see the model's output in the Streaming mode. In real-time interactions, low response time is crucial, but in offline scenarios, such as model evaluation, it is relatively less important.
- 2) **Time per output token (TPOT):** In the Streaming mode, once the model starts generating output, as long as the generation speed exceeds the human reading speed, it results in a very good experience. TPOT corresponds to the time it takes to generate each output token for a user during this phase. This metric aligns with how users perceive the model's "speed." For example, a TPOT of 100ms/token means that a user receives 10 tokens per second, or approximately 450 words per minute (assuming one word corresponds to 4/3 tokens). This rate is faster than the average reading speed of a person.
- 3) **Latency:** Here, latency refers to the total time required for the model to generate a complete response for the user. The overall response latency can be calculated using the first two metrics:

$$\text{Latency} = \text{TTFT} + \text{TPOT}$$

$$* (\text{Number of Tokens to be Generated} - 1). \quad (9)$$

Latency focuses on service experience: the faster the results are returned, the better the user experience. Latency optimization primarily involves low-level operations such as operator optimization, matrix optimization, parallel processing, and more efficient C++ inference. While optimizing latency can improve throughput, it is not as significant as directly increasing the batch size.

- 4) **Throughput:** Throughput refers to the rate at which output tokens are generated per second by the inference service for all user requests. Optimizing throughput focuses on system cost: the more the system processes in a given time, the higher its utilization. To enhance throughput, the primary method is KV cache access optimization, which reduces memory overhead and allows for a larger batch size.

2) TRADE-OFFS BETWEEN LATENCY AND THROUGHPUT

During high-request concurrency, batching user requests can improve throughput, but it may somewhat harm each user's latency. This happens because, instead of processing a single request, multiple requests are now processed together, leading to longer wait times for each user. Once a certain batch size is reached, and we enter a compute-bound state, further doubling the batch size will only increase latency without improving throughput. Typically, throughput increases with batch size, and latency also rises accordingly. As long as latency remains within an acceptable range, this trade-off is manageable. For some offline scenarios, we are not sensitive to latency and focus more on throughput. However, for real-time interactive applications, we must consider both latency and throughput.

The trade-off between latency and throughput fundamentally stems from the competition for computational and memory resources. Lowering inference latency necessitates minimizing computational demand while boosting inference throughput demands significant memory resources. Consequently, optimization techniques for LLM inference primarily aim to reduce both computational and memory requirements. The following subsections provide an overview of widely adopted techniques for optimizing computation and memory.

B. MODEL COMPRESSION

Model compression techniques provide important methods for optimizing inference by reducing the parameter size and computational and memory demands of LLMs. The basic motivation for model compression is that current models are redundant and can be downsized with minimal loss in accuracy. This subsection will introduce several key model compression techniques, including quantization, sparsity, and knowledge distillation.

1) QUANTIZATION

As transformer-based language models scale up rapidly, the demand for GPU memory and computational power increases significantly. It becomes increasingly important to reduce model storage size and improve computational efficiency. The size of a model is determined by its number of parameters and their precision, which is typically float32, float16, or bfloat16.

During training, to ensure accuracy, the main weights are always kept in FP32. However, during inference, FP16 weights usually provide similar accuracy to FP32, meaning

that using FP16 weights during inference can reduce GPU memory usage by half while maintaining the same results. So, can memory consumption be reduced further? The answer is yes, using quantization techniques, with INT8 quantization being the most common. There are two common methods for quantization:

- 1) **Quantization-aware training (QAT):** is a fine-tuning of the model, where the model is further trained with quantization in mind [81]. The quantization process is incorporated into the training process, allowing the model to be trained to retain its accuracy even after quantization, leading to benefits during deployment (lower latency, smaller model size, lower memory footprint).
- 2) **Post-training quantization (PTQ):** In PTQ [82], the model's weights and activations are evaluated on a representative dataset to determine the range of values taken by these parameters. These ranges are then used to quantize the weights and activations to the desired integer precision.

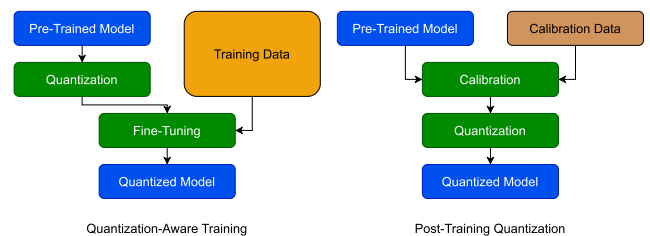


FIGURE 12. The illustration of QAT and PTQ.

When using LLMs to generate tokens with a small batch size, the process is limited by GPU memory bandwidth. The generation speed depends on how quickly model parameters can be transferred from GPU memory to the on-chip cache. Converting model weights from FP16 (2 bytes) to INT8 (1 byte) or INT4 (0.5 bytes) reduces the amount of data that needs to be moved, thereby speeding up token generation. However, quantization may negatively impact the quality of model outputs. The following methods are widely used in practice which give a high quantization rate while maintaining the model output quality:

- 1) **LLM.int8 quantization:** LLM.int8() [83] is primarily designed to lower the deployment threshold of large models without compromising performance. The BLOOM-176B model using LLM.int8() is approximately 15% to 23% slower than the FP16 version. Bitsandbytes is the CUDA-based library that mainly supports LLM.int8().
- 2) **SmoothQuant:** As model size increases, the range of values for individual tokens becomes larger, making quantization difficult. In contrast, the range of weight values is smaller, making weights easier to quantize while activations are harder to quantize. Based on this, the core idea of SmoothQuant [84] is to introduce a hyperparameter that reduces the range of activation

values and increases the range of weight values, thereby balancing the quantization difficulty between the two.

- 3) **GPTQ**: GPTQ [85] is a Layerwise Quantization algorithm. GPTQ quantizes the weights of the LLM one by one in isolation. GPTQ converts the floating-point parameters of each weight matrix into quantized integers such that the error at the output is minimized.

2) SPARSITY

Sparsity refers to a model having a very large capacity, but only certain parts of the model are activated for a given task, sample, or label. This allows for a significant increase in the model's capacity and capability without a proportional increase in computational load. An important method for achieving sparsity is pruning. Model quantization refers to compressing a model by reducing the number of bits required to represent weights or activations. Model pruning involves studying the redundancies in model weights and attempting to remove or prune redundant and non-essential weights.

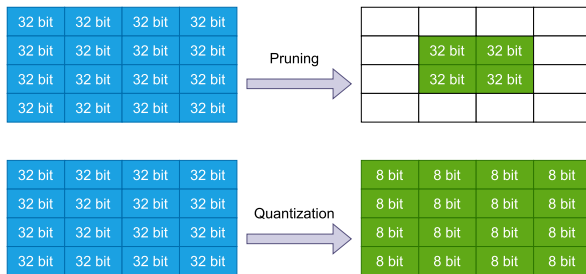


FIGURE 13. The illustration of model pruning and quantization.

Pruning reduces the model size by removing less important weights or connections while preserving the model's ability. It may or may not require re-training. Pruning can be either unstructured or structured:

- 1) **Unstructured pruning** [86] allows for the removal of any weights or connections, thus not preserving the original network architecture. Unstructured pruning is often not suitable for modern hardware and does not provide practical inference acceleration.
- 2) **Structured pruning** [87] aims to maintain a dense matrix multiplication form with certain elements set to zero. It may need to follow specific pattern constraints to be compatible with hardware kernel support. Current mainstream approaches focus on structured pruning to achieve high sparsity in Transformer models.

The following sparsification methods are widely used and studied, including, but are not limited to:

- 1) **SparseGPT**: This method [88] simplifies the pruning problem into a large-scale sparse regression instance. It is based on a new approximate sparse regression solver that addresses the hierarchical compression problem efficiently enough to perform on the largest GPT models (175B parameters) within a few hours using a single GPU. SparseGPT's accuracy is high enough that no

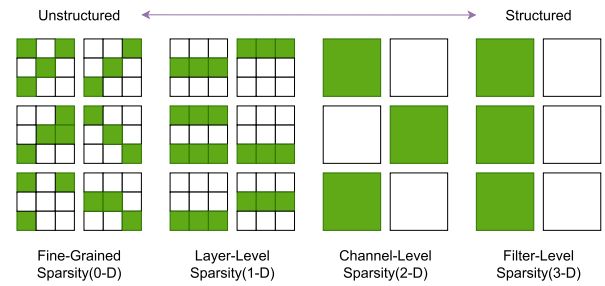


FIGURE 14. The illustration of unstructured and structured pruning.

fine-tuning is required, and the accuracy loss after pruning is negligible.

- 2) **LLM-Pruner**: This method [89] follows the classic 'importance estimation-pruning-fine-tuning' strategy, allowing large language models to be compressed with limited resources. Results show that even after pruning 20% of the parameters, the compressed model retains 93.6% of its performance.
- 3) **Wanda**: This method [90] consists of two simple but essential components—pruning metric and pruning granularity. The pruning metric assesses the importance of weights, which are then pruned according to the pruning granularity. This method can complete pruning on a 65B model in just 5.6 seconds, achieving results similar to SparseGPT.

3) KNOWLEDGE DISTILLATION

Model knowledge distillation (KD) is also an effective model compression strategy, as shown in Fig 12. It mainly consists of two types: white-box distillation [91] and black-box distillation [92]. White-box distillation focuses on the transparent handling of the model's internal structure and operational mechanisms, while black-box distillation does not require attention to the internal details of the source model. In practical applications, black-box distillation is more common. For example, many instruction-tuned models are fine-tuned using data generated by ChatGPT or GPT-4, which is a typical example of black-box distillation.

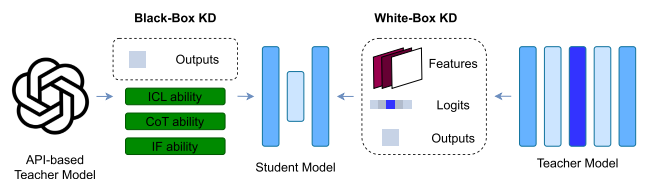


FIGURE 15. The illustration of white-box KD (right) and black-box KD (left).

In the process of model distillation, all operations are concentrated in the training phase of the model. The key during this phase is to leverage the knowledge of the large model (teacher model) to guide the learning of the smaller model (student model), enabling the smaller model to replicate the performance of the larger model while

maintaining a reduced size. It is important to note that the distillation process is transparent to the model's inference phase, meaning that after the distillation process is complete, the inference operations of the smaller model remain unaffected and can utilize all common inference optimization techniques. Although model distillation offers an effective means of compressing models while retaining performance, it requires generating and processing large amounts of data to train the student model and may involve multiple iterations to optimize the student model. Therefore, when considering the adoption of model distillation, it is necessary to weigh the resource investment against performance requirements to determine the most appropriate implementation strategy.

Since black-box distillation is more common and widely studied, we here briefly introduce three main approaches to black-box distillation:

- 1) **Chain-of-thought (CoT) distillation:** CoT distillation prompts large language models (LLMs) to generate intermediate reasoning steps, enabling them to tackle complex reasoning tasks step by step. References [93] and [94] use CoT techniques to enhance the reasoning capabilities of smaller models and improve their ability to generate explanations through a multi-task learning framework. This method can effectively transfer the reasoning capabilities of LLMs, improving the performance of smaller models on complex tasks.
- 2) **In-context learning (ICL) distillation:** ICL distillation uses structured prompts with task descriptions and examples to enable LLMs to learn new tasks without gradient updates. Reference [95] proposes a method that combines ICL objectives with language modeling objectives, training the student model to improve its generalization across various tasks by imitating the soft label predictions of the teacher model and the hard label ground truth values. This method can enhance the generalization ability of smaller models on multiple tasks without the need for retraining.
- 3) **Instruction following (IF) distillation:** IF distillation enhances the zero-shot ability of LLMs through fine-tuning using a collection of instruction-like prompt-response pairs. Reference [96] proposes Lion, which prompts the LLM to identify and generate "hard" instructions to enhance the capabilities of the student model; [97] develops a large collection of instructions and fine-tunes various models using these instructions. This method can significantly improve the performance of smaller models on zero-shot tasks, enabling them to better handle new tasks.

These three methods of black-box distillation each have their characteristics, but they all share the common goal of enhancing the performance and generalization ability of smaller models by extracting knowledge from large LLMs.

C. COMPUTATION OPTIMIZATION

The inference process of large models involves significant computational resources. Therefore, employing

computational optimization techniques during inference can significantly improve efficiency, reduce latency, and save hardware resources.

1) OPERATOR FUSION

Operator fusion is also a commonly used optimization technique in the inference process of large models (we have already briefly introduced the definition and benefits of operator fusion in training computation optimization). The core idea is to merge multiple operators (operations) into one during execution, in order to reduce data transmission, decrease memory access, and improve computational efficiency. Below is a detailed introduction to operator fusion technology:

1) Basic concept of operator fusion

- a) **Definition:** Operator fusion refers to the process of merging multiple independent operators (such as addition, multiplication, convolution, etc.) in the computation graph into a single operator during the inference process of deep learning models to optimize execution efficiency.
- b) **Motivation:** During the inference of deep learning models, data usually needs to be passed between different operators. If these operators are executed separately, it can lead to unnecessary data transmission and memory access, thereby increasing inference latency and computational overhead.

2) Types of operator fusion

- a) **Static fusion:** Before model compilation or deployment, some common operators in the computation graph are pre-merged. For example, merging the convolution operation and activation function into one operator. This method is simple and effective in many scenarios.
- b) **Dynamic fusion:** Depending on the characteristics of the input data and execution environment, dynamic fusion determines which operators can be merged during inference. This method is more flexible but complex to implement.

3) Common operator fusion strategies

- a) **Element-wise fusion:** Merging a series of element-wise operations (such as ReLU, addition, multiplication, etc.) into a single operation to reduce the storage and loading of intermediate results. For example, $Z = \text{ReLU}(X + Y)$ can be directly fused into one operation.
- b) **Convolution and activation function fusion:** Typically, an activation function (such as ReLU) follows a convolution operation, and these two operations can be merged into one to reduce the storage and loading of intermediate data.
- c) **Matrix multiplication and addition fusion:** In fully connected layers, matrix multiplication, and bias addition can be merged into one operation to reduce computational and memory overhead.

4) Implementation methods of operator fusion

- a) **Just-in-time compilation (JIT):** Through JIT compilation, operators are dynamically identified and fused during inference. For example, PyTorch's TorchScript and TensorFlow's XLA (Accelerated Linear Algebra) support this method.
- b) **Manual graph optimization:** By manually analyzing and modifying the computation graph, operators that can be fused are merged manually. This method is common in the low-level optimization of deep learning frameworks.
- c) **Framework-level support:** Some deep learning frameworks provide automatic operator fusion functionalities, such as TensorFlow's automatic graph optimizer and ONNX's graph optimization features.

5) Inference framework examples

- a) **FasterTransformer by NVIDIA** employs highly optimized GPU operator fusion techniques, particularly for the self-attention and feed-forward network layers within Transformer models. These fusions reduce data transfer and significantly improve computational efficiency. For example, in the self-attention mechanism, FasterTransformer merges the computation of queries, keys, and values with matrix multiplications into a single GPU kernel, minimizing intermediate data transfers and memory access.
- b) **DeepSpeed Inference by Microsoft** implements both dynamic and static operator fusion. It optimizes the computational graph of Transformer models by merging common operators such as element-wise operations and matrix multiplications, thereby reducing memory overhead and computation latency. It also supports graph-based operator fusion, which involves reordering and optimizing the model's computational graph during inference to achieve optimal computational performance, especially in low-latency, high-throughput scenarios.
- c) **MLC LLM by TVM** leverages TVM's compilation optimization techniques, which include automated graph optimizations and operator fusion to generate efficient inference code tailored to different hardware platforms. TVM supports static operator fusion, such as fusing convolution and activation functions, and performs operator-level optimizations during the compilation process. These optimizations not only enhance model inference speed but also generate specialized fused operators based on the characteristics of different hardware architectures.
- d) **TensorRT by NVIDIA** implements operator fusion through layer- and graph-level optimizations, particularly by merging multiple simple operations (such as convolution, batch normalization, and activation functions) into a single complex operation to maximize GPU computational efficiency. TensorRT also supports custom operator fusion, allowing developers to use APIs to fuse specific operators to meet the

requirements of particular applications. This flexibility enables TensorRT to deeply optimize inference tasks for various needs.

- e) **TurboTransformers by Tencent** employs lightweight operator fusion strategies, optimizing matrix multiplications and element-wise operations to reduce memory transfer and cache access, significantly enhancing inference speed. It also integrates hardware-specific optimized operators to ensure optimal inference performance across different devices (such as CPUs and GPUs). These fusion strategies make TurboTransformers perform exceptionally well in various inference scenarios.

These inference frameworks utilize operator fusion technology to optimize the computational efficiency of large model inference, reduce latency, and make the most of hardware resources. While each framework has its own unique fusion strategies and implementations, they all share the common goal of accelerating the model inference process by minimizing unnecessary computations and data transfers.

2) DISTRIBUTED INFERENCE

In the inference process of large models, due to the enormous scale of these models, a single computing device often struggles to meet the computational and memory demands. Therefore, distributed inference techniques have become crucial for improving inference efficiency and scaling model capabilities. Tensor Parallelism and Pipeline Parallelism are two common distributed parallel techniques, and here's a detailed introduction to each:

Tensor parallelism is a method of distributing a single tensor operation across multiple computing devices (such as GPUs). By splitting the model's computation tasks into smaller parts and executing them simultaneously on different devices, the inference process can be accelerated. It works in three steps:

- 1) **Model partitioning:** Large tensors (such as weight matrices) in the model are split into smaller sub-tensors either by rows or columns and assigned to different devices. For example, in a fully connected layer, the weight matrix can be split by columns, with each GPU responsible for computing part of the output.
- 2) **Parallel computation:** Each device performs local computation independently, followed by communication operations (such as All-Reduce, Gather, etc.) to combine the results. For instance, in matrix multiplication, each device computes part of the result, and then these partial results are aggregated to obtain the final output.
- 3) **Synchronization and communication:** To ensure consistency in parallel computation, synchronization and data transfer between devices are necessary, typically achieved through high-speed interconnects like NVLink.

The advantages of tensor parallelism:

- 1) **Scalability:** Effectively utilizes multiple GPU resources, enhancing computational power.
- 2) **High memory efficiency:** Each device only needs to store part of the model's weights, reducing the memory burden on a single device.
- 3) **Accelerated inference:** Significantly reduces inference time through parallel computation.

The challenges of tensor parallelism:

- 1) **Communication overhead:** The partitioned data needs to be transferred between devices, and communication latency can become a bottleneck.
- 2) **Load balancing:** Ensuring that computational tasks are balanced across devices to avoid overloading certain devices.
- 3) **Implementation complexity:** Requires appropriate partitioning of the model and parallel computation scheduling, adding to the complexity of implementation.

Pipeline parallelism is a method of distributing different layers or modules of a model across multiple computing devices and processing input data in a pipeline fashion. By assigning different stages of the inference process to different devices, overall inference throughput is improved. It works in three steps:

- 1) **Model partitioning:** The model is split into several parts by layers or modules, with each part assigned to a different device. For example, the first half of the layers might run on GPU A and the second half on GPU B.
- 2) **Pipeline processing:** Input data is passed sequentially through each device, with each device processing different batches of data simultaneously. For instance, while device A processes the n th batch of data, device B processes the $(n-1)$ th batch.
- 3) **Parallel execution:** By overlapping the processing of different batches, device utilization is maximized, reducing the overall inference time.

The advantages of pipeline parallelism:

- 1) **Increased throughput:** Significantly improves overall inference throughput by parallel processing multiple batches of data.
- 2) **Reduced latency:** Although the latency of a single batch might slightly increase, the overall system throughput improves, making it suitable for high-concurrency scenarios.
- 3) **High resource utilization:** Each device processes different data at different times, fully utilizing computing resources.

The challenges of pipeline parallelism:

- 1) **Batch scheduling:** Requires precise scheduling of batch processing on each device to avoid resource idling or conflicts.
- 2) **Model partitioning strategy:** Determining how to partition the model across devices to minimize communication and synchronization overhead.

- 3) **Complex synchronization mechanism:** Efficient data transfer and synchronization between devices are needed to ensure smooth pipeline operation.

In practice, tensor parallelism and pipeline parallelism are often used together to fully exploit multi-GPU resources. For example, tensor parallelism can be used within each pipeline stage to further increase parallelism and computational efficiency. This hybrid parallelism strategy can provide greater advantages in large-scale model inference. For examples:

- 1) **NVIDIA Megatron-LM:** Combines tensor parallelism and pipeline parallelism to support ultra-large-scale transformer model inference.
- 2) **Microsoft DeepSpeed:** Offers various parallel strategies, including Tensor Parallelism, Pipeline Parallelism, and Data Parallelism, which can be flexibly combined to optimize inference performance.
- 3) **TensorFlow Mesh:** Supports complex distributed parallel strategies, enabling the simultaneous use of Tensor Parallelism and Pipeline Parallelism.

Tensor Parallelism and Pipeline Parallelism are two essential distributed parallel techniques in large model inference. Tensor Parallelism improves computational power and memory efficiency by splitting tensor operations across multiple devices; Pipeline Parallelism enhances inference throughput by distributing different parts of the model across devices and processing data in a pipeline manner. These techniques have their respective advantages and challenges and are often combined to achieve efficient large model inference.

D. MEMORY OPTIMIZATION

Managing memory efficiently is a key challenge in serving LLMs, primarily due to the memory-intensive demands of transformer architectures. As the need for long-sequence inference grows, the memory usage of the KV cache has become a critical focus for optimization, surpassing concerns related to model weights and the workspace required for other activations. This subsection briefly introduces widely adopted memory optimization techniques, especially for KV cache management.

1) PAGEATTENTION

PageAttention is currently a crucial technique for optimizing the KV cache. The core of the highly popular large model inference acceleration project, vLLM [78], is based on PageAttention technology. In the cache, KV caches are large and their sizes are dynamically changing, making them difficult to predict. In existing systems, 60%-80% of the GPU memory is wasted due to memory fragmentation and excessive reservation. Page Attention offers a solution by leveraging a paging mechanism, similar to how operating systems manage memory. Instead of storing the entire attention matrix in memory, Page Attention divides the matrix into smaller, manageable pages. These pages are stored in a secondary storage (e.g., slower memory) and only the

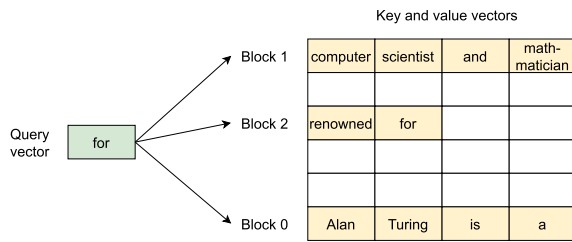


FIGURE 16. The illustration of how PagedAttention stores the attention key and values vectors as non-contiguous blocks in the memory.

relevant pages needed for computation are loaded into the faster, primary memory when required.

This approach drastically reduces the peak memory usage during inference, allowing for the processing of longer sequences without running into memory bottlenecks. The following steps show how Page Attention works:

- 1) **Matrix partitioning:** The attention matrix is divided into smaller chunks, or “pages,” that can be stored separately. This partitioning is typically done in a way that aligns with the computation patterns of the attention mechanism.
- 2) **Selective loading:** During the attention computation, only the necessary pages are loaded into the GPU memory. Pages are fetched and evicted as needed, minimizing the memory footprint.
- 3) **Efficient memory management:** By managing memory more efficiently, Page Attention reduces the need for frequent data transfers between different memory layers, improving both speed and efficiency during inference.

2) TREE ATTENTION

Tree Attention, as proposed in the SpecInfer [98], reimagines the self-attention mechanism by organizing the attention computation in a hierarchical tree structure. Instead of directly computing attention scores between every pair of tokens, Tree Attention groups tokens into clusters and computes attention within these clusters. These clusters are then recursively aggregated, forming a tree-like structure. Finally, Tree Attention apply depth-first tree traversal to eliminate redundant KV cache allocation for multiple output sequences sharing the same prefix.

This hierarchical approach reduces the number of attention computations required, significantly lowering the complexity of the operation. By carefully structuring the tree, Tree Attention can approximate the full attention mechanism with much less computational overhead, making it more efficient for large-scale models. The following steps show how Tree Attention works:

- 1) **Hierarchical clustering:** Tokens in the input sequence are first grouped into clusters based on their contextual similarity or other criteria. Each cluster represents a subtree in the overall attention tree.
- 2) **Recursive attention calculation:** Within each cluster, attention scores are computed among the tokens. These

scores are then aggregated and propagated up the tree, where higher-level clusters represent broader contexts.

- 3) **Selective attention:** As the attention scores are propagated up the tree, only the most relevant clusters (and their tokens) are retained for further computation. This selective attention reduces the number of attention operations while maintaining the model’s ability to capture important relationships.
- 4) **Efficiency gains:** By reducing the number of pairwise attention computations, Tree Attention dramatically lowers the computational and memory requirements of the self-attention mechanism, especially for long sequences.

3) GQA AND MQA

In large model inference, attention mechanisms are key components for enhancing model performance. This subsection will delve into three commonly used attention mechanisms: Multi-Head Attention (MHA), Grouped Query Attention (GQA), and Multi-Query Attention (MQA), discussing their functionality, performance, and differences.

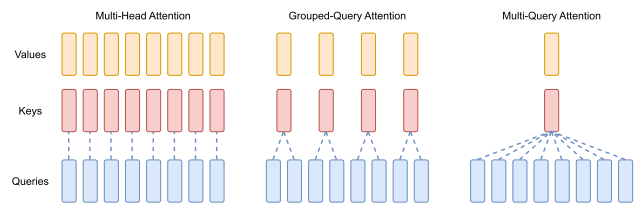


FIGURE 17. The illustration of MHA, GQA, and MQA.

Multi-Head Attention (MHA) [7] is a central part of the transformer architecture. MHA works by splitting the attention operation on the input sequence into multiple parallel “heads”, each independently learning to extract features from different subspaces. Specifically, each head performs a scaled dot-product attention calculation within its subspace, resulting in different attention distributions. Finally, the outputs of these heads are concatenated and linearly transformed to produce the final output.

The design of MHA allows the model to capture multiple dependencies between different positions in the input sequence simultaneously. This parallel processing increases the model’s expressive power, making MHA highly effective in capturing complex patterns and long-range dependencies. However, because MHA requires independent attention calculations for each head, it has a high computational complexity, especially during inference, which can lead to longer inference times and higher memory usage.

Grouped Query Attention (GQA) [99] is an optimization of MHA. It divides the query vectors into multiple groups, with each group attending only to a subset of key-value pairs. This approach reduces the computational load while retaining some flexibility of multi-head attention.

GQA reduces the computational complexity of MHA by limiting the attention calculation for each head, which

TABLE 3. The comparison of MHA, GQA, and MQA.

Feature	MHA	GQA	MQA
Computational Complexity	High	Moderate	Low
Memory Requirements	High	Moderate	Low
Inference Speed	Slow	Faster	Fast
Information Capturing Ability	Strong	Moderate	Weaker
Application Scenarios	Complex dependencies, multi-head parallel processing	Scenarios balancing performance and efficiency	Resource-constrained, high-efficiency inference

can significantly improve inference speed, especially in large-scale data tasks. However, because each group only interacts with a portion of the key-value pairs, there may be some information loss, potentially impacting model accuracy.

Multi-Query Attention (MQA) [100] further optimizes MHA by sharing keys and values, meaning that multiple query vectors use the same key-value pairs for attention computation. This approach significantly reduces the redundancy in MHA's calculations.

By sharing keys and values, MQA dramatically reduces computational and memory requirements, thus speeding up inference. The advantages of MQA are particularly pronounced in tasks involving long sequences or large-scale data processing. However, since multiple queries share the same key-value pairs, MQA might not capture attention distributions with the same granularity as MHA, which could affect model precision in some tasks. Table 3 lists the comparison and summary of MHA, GQA, and MQA:

In practical applications, the choice of attention mechanism should depend on the specific task requirements, system computing resources, and the need for inference speed. MHA provides the strongest information-capturing ability, making it suitable for tasks requiring complex contextual dependencies; GQA strikes a balance between performance and computational efficiency, making it ideal for large-scale data processing; while MQA greatly reduces computational overhead and excels in inference efficiency, making it suitable for resource-limited environments that require fast inference.

In summary, MHA, GQA, and MQA represent different strategies for memory(computational) trade-offs, and researchers and engineers can choose the most appropriate attention mechanism based on the specific application needs to optimize large model inference performance.

E. DECODING OPTIMIZATION

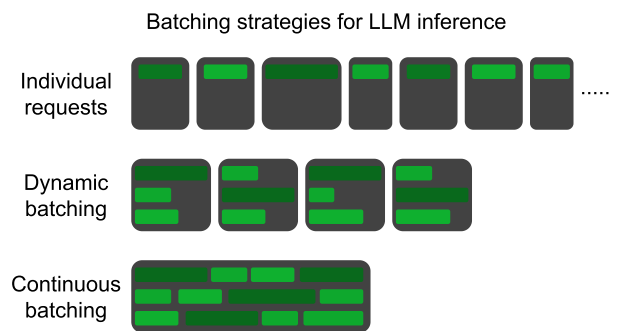
Decoding optimization in large model inference refers to techniques and strategies aimed at improving the efficiency and speed of generating outputs (decoding) from a model during inference. These optimizations focus on reducing computational overhead, minimizing latency, and enhancing throughput while maintaining the quality of the generated results. This subsection briefly introduces three effective in practices or promising optimization techniques:

continuous batching, speculative decoding, and blockwise parallel decoding, each designed to accelerate the decoding process in large-scale language models without significantly sacrificing performance.

1) CONTINUOUS BATCHING

Batch inference is essential for serving LLMs when using limited computation and memory resources. There are four ways inference requests can be batched on a GPU:

- 1) No batching: each request is processed one at a time.
- 2) Static batching: requests are placed in batches that are run when full.
- 3) Dynamic batching: requests are placed in batches as they're received and batches run once full or once enough time has elapsed since the first request.
- 4) Continuous batching: requests are processed token-by-token, with new requests getting processed as older requests finish and free up space on the GPU.


FIGURE 18. The illustration of continuous batching.

Continuous batching is an optimization strategy that builds upon the principles of dynamic batching but takes the concept further by continuously processing incoming requests as they arrive. Instead of waiting for a batch to fill up to a predetermined size or waiting for a time-based trigger, continuous batching ensures that as soon as a minimum viable batch is ready, it is immediately processed. This approach minimizes idle time and allows for a smoother flow of data through the inference pipeline.

The continuous batching system operates with several key components:

- 1) **Request stream:** In a high-concurrency environment, requests are constantly arriving at varying rates. These requests are typically of different sizes and may require different levels of computational resources.
- 2) **Batching logic:** Continuous batching involves sophisticated logic that monitors the incoming request stream in real-time. The system evaluates factors such as the current load, the available computational resources, and the urgency of the requests.
- 3) **Dynamic adjustment:** Unlike static or even dynamic batching, continuous batching does not rely on fixed batch sizes or strict scheduling algorithms. Instead, it dynamically adjusts batch sizes and schedules based on real-time conditions. This means that the system can process a few large requests immediately or aggregate multiple smaller requests into a batch within milliseconds, depending on the current context.
- 4) **Adaptive execution:** The inference model is continuously fed with new batches, which may vary in size and content. This continuous feeding mechanism ensures that the model is always utilized to its full potential without unnecessary delays, thus optimizing both throughput and latency.

Continuous batching offers several distinct advantages, particularly in environments that demand high responsiveness and scalability:

- 1) **Reduced latency:** By processing batches as soon as they are ready, continuous batching significantly reduces the latency associated with waiting for a batch to fill. This is especially important for applications requiring real-time or near-real-time responses.
- 2) **Increased throughput:** Continuous batching maximizes the utilization of computational resources by ensuring that the inference engine is constantly processing data, thus increasing the overall throughput of the system.
- 3) **Flexibility and adaptability:** Continuous batching is highly adaptable to varying loads and request patterns. It can handle sudden spikes in demand without the bottlenecks that static batching might introduce.
- 4) **Scalability:** In cloud or edge environments, continuous batching scales effectively with the available resources, ensuring that performance remains consistent even as the volume of incoming requests grows.

Continuous batching represents an advanced approach to optimizing large model inference, particularly in scenarios demanding both high throughput and low latency. By continuously adapting to real-time conditions and processing requests as they arrive, this technique ensures that the inference pipeline is always operating at peak efficiency. While it introduces some complexity in implementation and resource management, the benefits it offers in terms of responsiveness and scalability make it an invaluable tool in modern AI deployment strategies.

2) SPECULATIVE DECODING

Speculative decoding [101] is an advanced technique designed to accelerate the inference phase of LLMs. As LLMs become increasingly complex, the computational cost associated with generating text outputs—known as decoding—has escalated significantly. Speculative decoding offers a way to mitigate these costs by leveraging predictions made by smaller, faster models, thereby reducing the overall computational burden without compromising output quality.

Decoding is the process of generating sequences (e.g., text) from a model, often performed in an auto-regressive manner. In this context, each token in a sequence is generated one at a time, with each subsequent token depending on the previously generated tokens. While this method produces high-quality results, it can be computationally expensive, particularly when using large models. Two common decoding strategies are:

- 1) **Greedy decoding:** [102] Selects the most likely next token at each step, which is fast but may miss higher-quality sequences.
- 2) **Beam search:** [103] Explores multiple possible sequences in parallel, balancing speed and quality but at a higher computational cost.

Both approaches face scalability issues with large models, making them suitable candidates for optimization techniques like speculative decoding.

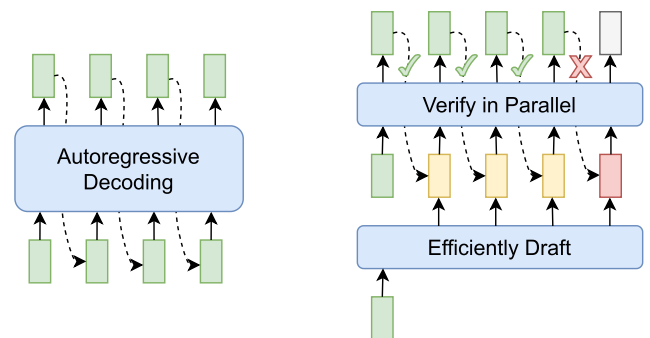


FIGURE 19. The illustration of speculative decoding.

Speculative decoding aims to improve inference efficiency by employing a smaller, approximate model (often referred to as the “draft” model) alongside the main large model. The process involves generating multiple candidate sequences using the draft model and then validating these sequences with the larger, more accurate model. The technique can be broken down into the following steps:

- 1) **Draft model prediction:** The draft model generates a set of potential sequences or partial sequences based on the current input context.
- 2) **Candidate validation:** The large model evaluates the candidates proposed by the draft model. If the sequences are deemed valid (e.g., if the likelihood of the sequences according to the large model is above a certain threshold), they are accepted.

3) **Fallback mechanism:** If the draft model's predictions are not validated by the large model, the system falls back to the traditional decoding process, where the large model generates the next tokens.

4) **Final output:** The validated sequences from the draft model are used as the final output, or the process iterates until a satisfactory sequence is produced.

The benefits of speculative decoding include:

- 1) **Speedup:** By offloading the bulk of the computation to a smaller model, speculative decoding can significantly reduce the time required for inference. The large model only needs to process a subset of the potential outputs, rather than generating every token itself.
- 2) **Resource efficiency:** Speculative decoding reduces the demand on computational resources, as the draft model is typically much less resource-intensive than the full model.
- 3) **Flexibility:** The technique can be adapted to various types of models and decoding strategies, making it a versatile tool in the context of large-scale AI deployment.

Speculative decoding represents a promising direction for improving the efficiency of large model inference. As the size and complexity of LLMs continue to grow, techniques like speculative decoding will become increasingly important for making these models more accessible and cost-effective. Future research may focus on developing more sophisticated draft models, improving the validation process, and exploring how speculative decoding can be combined with other optimization techniques to further enhance performance.

3) BLOCKWISE PARALLEL DECODING

Blockwise parallel decoding [104] addresses the sequential nature of traditional decoding by dividing the generation process into “blocks” of tokens that can be processed in parallel. This approach leverages the observation that dependencies within a block are relatively localized, allowing multiple tokens within the same block to be decoded simultaneously. The technique can be understood through the following steps:

- 1) **Predict phase:** Set $p_1 = p$ (where p represents the original model), and then additionally train a series of auxiliary models $p_2, \dots, p_k$, where $p_i(y_{j+i} | y_{\leq j}, x)$. In other words, the i -th model generates the $j + i$ -th output based on the previous 1 to j outputs. For example, if the current sequence of generated words is “I saw a dog ride,” the first model (original model) is responsible for generating the word at the position of “in,” the second model is responsible for generating the word at the position of “the,” and the third model is responsible for generating the word at the position of “bus.”
- 2) **Verify phase:** In the previous step, K new outputs were generated, but it is not yet known whether all K outputs are correct, so further verification is needed. The previous $j - 1$ new words are combined with the input, and the next word is predicted. If the next word is the same as the j -th word, then this word is accepted. For

example, if the previous step generated “in,” “the,” and “car,” then the model p_1 (original model) needs to verify these in three groups (which can be executed in parallel):

- a) **First group:** Input “I saw a dog ride,” word to be verified is “in,” the actual predicted Top 1 is “in,” results match, “in” is accepted.
 - b) **Second group:** Input “I saw a dog ride in,” word to be verified is “the,” the actual predicted Top 1 is “the,” results match, “the” is accepted.
 - c) **Third Group:** Input “I saw a dog ride in the,” word to be verified is “car,” the actual predicted Top 1 is “bus,” results do not match, “car” is not accepted.
- 3) **Accept phase:** Suppose the previous step generated 10 words, and there is an inconsistency at the 5th word. In that case, the first 4 words can be merged with the input, and the next generation iteration can begin. Referring back to the example in the previous two steps, because “car” and “bus” do not match in the third group, only “in” and “the” are accepted. The input for the next iteration is “I saw a dog ride in the.”

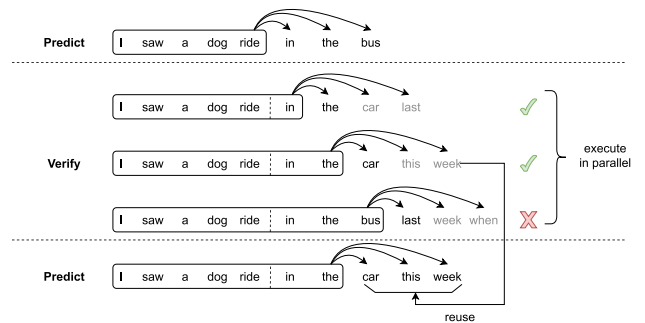


FIGURE 20. The illustration of blockwise parallel decoding.

The advantages of blockwise parallel decoding:

- 1) **Reduced latency:** By parallelizing token generation within blocks, blockwise parallel decoding can significantly reduce the time required for decoding, particularly for long sequences. This makes it well-suited for applications requiring low-latency responses.
- 2) **Increased throughput:** The ability to process multiple tokens simultaneously increases overall throughput, allowing the model to generate more sequences in a given time frame.
- 3) **Scalability:** Blockwise parallel decoding is particularly beneficial as the model size increases, as the potential for parallelism grows with the number of tokens generated per block.

Blockwise parallel decoding is a promising approach to overcoming the limitations of traditional, sequential decoding methods in large model inference. By leveraging parallelism within blocks of tokens, this technique offers a pathway to faster, more efficient generation of text and other outputs, making it an essential tool in the ongoing development and deployment of large language models.

VI. CONCLUSION

The pursuit of efficiency in LLM training and inference has catalyzed the development of a diverse array of innovative techniques and research directions. These include advances in model architectures, training algorithms, hardware acceleration, and adaptive computation. As LLMs continue to scale, these avenues of exploration will be crucial for enhancing their accessibility, cost-effectiveness, and environmental sustainability.

This paper has introduced several well-established and effective optimization techniques for LLM training and inference. Beyond the optimization techniques discussed in this paper, numerous promising research directions exist, including mixture-of-experts models, early exit mechanisms, and second-order optimization. By consolidating the latest research findings, this paper aims to accelerate progress and foster innovation in the pursuit of highly efficient LLM training and serving solutions.

REFERENCES

- [1] B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33. Red Hook, NY, USA: Curran Associates, 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [2] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
- [3] A. Jo, "The promise and peril of generative AI," *Nature*, vol. 614, no. 1, pp. 214–216, 2023.
- [4] X. L. Dong, S. Moon, Y. E. Xu, K. Malik, and Z. Yu, "Towards next-generation intelligent assistants leveraging LLM techniques," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 5792–5793.
- [5] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever, "Zero-shot text-to-image generation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2021, pp. 8821–8831.
- [6] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023, *arXiv:2310.08560*.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [8] X. Yang, H. Zhang, G. Qi, and J. Cai, "Causal attention for vision-language tasks," 2021, *arXiv:2103.03493*.
- [9] V. Bozic, D. Dordevic, D. Coppola, J. Thommes, and S. P. Singh, "Rethinking attention: Exploring shallow feed-forward neural networks as an alternative to attention layers in transformers," 2023, *arXiv:2311.10642*.
- [10] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," 2019, *arXiv:1904.10509*.
- [11] Z. Pang, Z. Xie, Y. Man, and Y.-X. Wang, "Frozen transformers in language models are effective visual encoder layers," 2023, *arXiv:2310.12973*.
- [12] T. Wang, A. Roberts, D. Hesslow, T. Le Scao, H. W. Chung, I. Beltagy, J. Launay, and C. Raffel, "What language model architecture and pretraining objective work best for zero-shot generalization?" 2022, *arXiv:2204.05832*.
- [13] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," 2022, *arXiv:2204.02311*.
- [14] S. Wu, X. Zhao, T. Yu, R. Zhang, C. Shen, H. Liu, F. Li, H. Zhu, J. Luo, L. Xu, and X. Zhang, "Yuan 1.0: Large-scale pre-trained language model in zero-shot and few-shot learning," 2021, *arXiv:2110.04725*.
- [15] V. Sanh et al., "Multitask prompted training enables zero-shot task generalization," 2021, *arXiv:2110.08207*.
- [16] J. W. Rae et al., "Scaling language models: Methods, analysis & insights from training gopher," 2021, *arXiv:2112.11446*.
- [17] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," 2019, *arXiv:1910.10683*.
- [18] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 3, pp. 111–132, Jan. 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666651022000146>
- [19] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 12, pp. 11106–11115.
- [20] Y. Xiong, Z. Zeng, R. Chakraborty, M. Tan, G. Fung, Y. Li, and V. Singh, "Nyströmformer: A Nyström-based algorithm for approximating self-attention," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 16, pp. 14138–14148.
- [21] B. Wang, L. Shang, C. Lioma, X. Jiang, H. Yang, Q. Liu, and J. G. Simonsen, "On position embeddings in BERT," in *Proc. Int. Conf. Learn. Represent.*, 2020.
- [22] B. Wang, Z. Donghao, L. Christina, Q. Li, Z. Peng, and J. G. Simonsen, "Encoding word order in complex embeddings," in *Proc. 8th Int. Conf. Learn. Represent.*, 2020.
- [23] J. Xu, X. Sun, Z. Zhang, G. Zhao, and J. Lin, "Understanding and improving layer normalization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [24] T. Bachlechner, B. P. Majumder, H. H. Mao, G. W. Cottrell, and J. McAuley, "ReZero is all you need: Fast convergence at large depth," 2020, *arXiv:2003.04887*.
- [25] A. Yang, J. Lin, R. Men, C. Zhou, L. Jiang, X. Jia, A. Wang, J. Zhang, J. Wang, Y. Li, D. Zhang, W. Lin, L. Qu, J. Zhou, and H. Yang, "M6-T: Exploring sparse expert models and beyond," 2021, *arXiv:2105.15082*.
- [26] S. Sukhbaatar, E. Grave, G. Lample, H. Jegou, and A. Joulin, "Augmenting self-attention with persistent memory," 2019, *arXiv:1907.01470*.
- [27] Z. Wu, Z. Liu, J. Lin, Y. Lin, and S. Han, "Lite transformer with long-short range attention," 2020, *arXiv:2004.11886*.
- [28] Y. Wang, Y. Yang, J. Bai, M. Zhang, J. Bai, J. Yu, C. Zhang, and Y. Tong, "Predictive attention transformer: Improving transformer with attention map prediction," in *Proc. ICLR*, 2021.
- [29] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin, "DeeBERT: Dynamic early exiting for accelerating BERT inference," 2020, *arXiv:2004.12993*.
- [30] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-XL: Attentive language models beyond a fixed-length context," 2019, *arXiv:1901.02860*.
- [31] Y. Zhao, L. Dong, Y. Shen, Z. Zhang, F. Wei, and W. Chen, "Memory-efficient differentiable transformer architecture search," 2021, *arXiv:2105.14669*.
- [32] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [33] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big bird: Transformers for longer sequences," 2020, *arXiv:2007.14062*.
- [34] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," Tech. Rep., 2018.
- [35] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019, *arXiv:1910.13461*.
- [36] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
- [37] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," 2021, *arXiv:2101.03961*.
- [38] D. So, Q. Le, and C. Liang, "The evolved transformer," in *Proc. Int. Conf. Mach. Learn.*, vol. 97, 2019, pp. 5877–5886.
- [39] N. Parmar, A. Vaswani, J. Uszkoreit, L. Kaiser, N. Shazeer, A. Ku, and D. Tran, "Image transformer," 2018, *arXiv:1802.05751*.
- [40] L. Dong, S. Xu, and B. Xu, "Speech-transformer: A no-recurrence sequence-to-sequence model for speech recognition," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2018, pp. 5884–5888.

- [41] L. Harold Li, M. Yatskar, D. Yin, C.-J. Hsieh, and K.-W. Chang, "VisualBERT: A simple and performant baseline for vision and language," 2019, *arXiv:1908.03557*.
- [42] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
- [43] S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhume, G. Zerveas, V. Korthikanti, E. Zhang, R. Child, R. Y. Aminabadi, J. Bernauer, X. Song, M. Shoenybi, Y. He, M. Houston, S. Tiwary, and B. Catanzaro, "Using DeepSpeed and megatron to train megatron-turing NLG 530B, a large-scale generative language model," 2022, *arXiv:2201.11990*.
- [44] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," 2019, *arXiv:1909.08053*.
- [45] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan. (2022). *Accelerate: Training and Inference At Scale Made Simple, Efficient and Adaptable*. [Online]. Available: <https://github.com/huggingface/accelerate>
- [46] S. Li, H. Liu, Z. Bian, J. Fang, H. Huang, Y. Liu, B. Wang, and Y. You, "Colossal-AI: A unified deep learning system for large-scale parallel training," 2021, *arXiv:2110.14883*.
- [47] FairScale authors. (2021). *FairScale: A General Purpose Modular Pytorch Library for High Performance and Large Scale Training*. [Online]. Available: <https://github.com/facebookresearch/fairscale>
- [48] R. Bi, T. Xu, M. Xu, and E. Chen, "PaddlePaddle: A production-oriented deep learning platform facilitating the competency of enterprises," in *Proc. IEEE 24th Int. Conf. High Perform. Comput. Commun.; 8th Int. Conf. Data Sci. Syst.; 20th Int. Conf. Smart City; 8th Int. Conf. Dependability Sensor, Cloud Big Data Syst. Appl. (HPCC/DSS/SmartCity/DependSys)*, Dec. 2022, pp. 92–99.
- [49] R. Y. Aminabadi, S. Rajbhandari, M. Zhang, A. A. Awan, C. Li, D. Li, E. Zheng, J. Rasley, S. Smith, O. Ruwase, and Y. He, "DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale," 2022, *arXiv:2207.00032*.
- [50] J. Kim, J. Lee, J. Park, D. Min, and K. Sohn, "Pin the memory: Learning to generalize semantic segmentation," 2022, *arXiv:2204.03609*.
- [51] J. Dong, Z. Cao, T. Zhang, J. Ye, S. Wang, F. Feng, L. Zhao, X. Liu, L. Song, L. Peng, Y. Guo, X. Jiang, L. Tang, Y. Du, Y. Zhang, P. Pan, and Y. Xie, "EFLOPS: Algorithm and system co-design for a high performance distributed training platform," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 610–622.
- [52] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "DNNFusion: Accelerating deep neural networks execution with advanced operator fusion," in *Proc. 42nd ACM SIGPLAN Int. Conf. Program. Lang. Design Implement.*, Jun. 2021, pp. 883–898.
- [53] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," 2022, *arXiv:2205.14135*.
- [54] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," 2023, *arXiv:2307.08691*.
- [55] V. Klema and A. Laub, "The singular value decomposition: Its computation and some applications," *IEEE Trans. Autom. Control*, vol. AC-25, no. 2, pp. 164–176, Apr. 1980.
- [56] D. Guo, A. Rush, and Y. Kim, "Parameter-efficient transfer learning with diff pruning," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process.*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds., 2021, pp. 4884–4896. [Online]. Available: <https://aclanthology.org/2021.acl-long.378>
- [57] M. Zhao, T. Lin, F. Mi, M. Jaggi, and H. Schütze, "Masking as an efficient alternative to finetuning for pretrained language models," 2020, *arXiv:2004.12406*.
- [58] K. Egashira, M. Vero, R. Staab, J. He, and M. Vechev, "Exploiting LLM quantization," 2024, *arXiv:2405.18137*.
- [59] X. Xu, M. Li, C. Tao, T. Shen, R. Cheng, J. Li, C. Xu, D. Tao, and T. Zhou, "A survey on knowledge distillation of large language models," 2024, *arXiv:2402.13116*.
- [60] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," 2021, *arXiv:2106.10199*.
- [61] T. Vu, B. Lester, N. Constant, R. Al-Rfou, and D. Cer, "SPoT: Better frozen model adaptation through soft prompt transfer," 2021, *arXiv:2110.07904*.
- [62] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process.*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds., Aug. 2021, pp. 4582–4597. [Online]. Available: <https://aclanthology.org/2021.acl-long.353>
- [63] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," 2017, *arXiv:1710.03740*.
- [64] P. V. Kotipalli, R. Singh, P. Wood, I. Laguna, and S. Bagchi, "AMPT-GA: Automatic mixed precision floating point tuning for GPU applications," in *Proc. ACM Int. Conf. Supercomput.*, Jun. 2019, pp. 160–170.
- [65] J. Lamy-Poirier, "Layered gradient accumulation and modular pipeline parallelism: Fast and efficient training of large language models," 2021, *arXiv:2106.02679*.
- [66] P. Jain, A. Jain, A. Nrusimha, A. Gholami, P. Abbeel, J. Gonzalez, K. Keutzer, and I. Stoica, "CheckMate: Breaking the memory wall with optimal tensor rematerialization," in *Proc. Mach. Learn. Syst.*, vol. 2, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., 2020, pp. 497–511. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/file/0b816ae8f06f8dd3543dc3d9ef196cab-Paper.pdf
- [67] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," 2019, *arXiv:1910.02054*.
- [68] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-infinity: Breaking the GPU memory wall for extreme scale deep learning," 2021, *arXiv:2104.07857*.
- [69] G. Wang, H. Qin, S. A. Jacobs, C. Holmes, S. Rajbhandari, O. Ruwase, F. Yan, L. Yang, and Y. He, "ZeRO++: Extremely efficient collective communication for giant model training," 2023, *arXiv:2306.10209*.
- [70] S. Dash, I. Lyngaas, J. Yin, X. Wang, R. Egele, G. Cong, F. Wang, and P. Balaprakash, "Optimizing distributed training on frontier for large language models," 2023, *arXiv:2312.12705*.
- [71] C. Luo, T. Zhong, and G. Fox, "RTP: Rethinking tensor parallelism with memory deduplication," 2023, *arXiv:2311.01635*.
- [72] A. Sun, W. Zhao, X. Han, C. Yang, X. Zhang, Z. Liu, C. Shi, and M. Sun, "Seq1F1B: Efficient sequence-level pipeline parallelism for large language model training," 2024, *arXiv:2406.03488*.
- [73] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, G. Dai, X.-P. Zhang, Y. Dong, and Y. Wang, "A survey on efficient inference for large language models," 2024, *arXiv:2404.14294*.
- [74] Z. Wan, X. Wang, C. Liu, S. Alam, Y. Zheng, J. Liu, Z. Qu, S. Yan, Y. Zhu, Q. Zhang, M. Chowdhury, and M. Zhang, "Efficient large language models: A survey," 2023, *arXiv:2312.03863*.
- [75] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," 2022, *arXiv:2211.05102*.
- [76] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, H. Jin, T. Chen, and Z. Jia, "Towards efficient generative large language model serving: A survey from algorithms to systems," 2023, *arXiv:2312.15234*.
- [77] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko, and Y. He, "DeepSpeed-FastGen: High-throughput text generation for LLMs via MII and DeepSpeed-inference," 2024, *arXiv:2401.08671*.
- [78] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ.*, Oct. 2023, pp. 611–626.
- [79] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, Y. Dong, and Y. Wang, "FlashDecoding++: Faster large language model inference on GPUs," 2023, *arXiv:2311.01282*.
- [80] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," 2023, *arXiv:2308.07633*.
- [81] Y. Xu, X. Han, Z. Yang, S. Wang, Q. Zhu, Z. Liu, W. Liu, and W. Che, "OneBit: Towards extremely low-bit large language models," 2024, *arXiv:2402.11295*.
- [82] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "OmniQuant: Omnidirectionally calibrated quantization for large language models," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024. [Online]. Available: <https://openreview.net/forum?id=8Wuvh0LYW>

- [83] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” 2022, *arXiv:2208.07339*.
- [84] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” 2022, *arXiv:2211.10438*.
- [85] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” 2022, *arXiv:2210.17323*.
- [86] Y. Zhang, L. Zhao, M. Lin, S. Yunyun, Y. Yao, X. Han, J. Tanner, S. Liu, and R. Ji, “Dynamic sparse no training: Training-free fine-tuning for sparse LLMs,” in *Proc. 12th Int. Conf. Learn. Represent.*, 2024. [Online]. Available: <https://openreview.net/forum?id=1ndDmZdT4g>
- [87] M. Xia, T. Gao, Z. Zeng, and D. Chen, “Sheared LLaMA: Accelerating language model pre-training via structured pruning,” in *Proc. 12th Int. Conf. Learn. Represent.*, 2024. [Online]. Available: <https://openreview.net/forum?id=09iOdaeOzp>
- [88] E. Frantar and D. Alistarh, “SparseGPT: Massive language models can be accurately pruned in one-shot,” in *Proc. 40th Int. Conf. Mach. Learn.*, in Proceedings of Machine Learning Research, vol. 202, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., Jul. 2023, pp. 10323–10337. [Online]. Available: <https://proceedings.mlr.press/v202/frantar23a.html>
- [89] X. Ma, G. Fang, and X. Wang, “LLM-pruner: On the structural pruning of large language models,” in *Proc. 37th Conf. Neural Inf. Process. Syst.*, 2023. [Online]. Available: <https://openreview.net/forum?id=J8Ajf9WfXP>
- [90] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “A simple and effective pruning approach for large language models,” in *Proc. 12th Int. Conf. Learn. Represent.*, 2024. [Online]. Available: <https://openreview.net/forum?id=PxoFut3dWW>
- [91] Y. Gu, L. Dong, F. Wei, and M. Huang, “MiniLLM: Knowledge distillation of large language models,” in *Proc. 12th Int. Conf. Learn. Represent.*, 2024. [Online]. Available: <https://openreview.net/forum?id=5h0qf7IBZZ>
- [92] Y. Li, P. Yuan, S. Feng, B. Pan, B. Sun, X. Wang, H. Wang, and K. Li, “Turning dust into gold: Distilling complex reasoning capabilities from LLMs by leveraging negative data,” in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 17, pp. 18591–18599. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/29821>
- [93] S. Li, J. Chen, Y. Shen, Z. Chen, X. Zhang, Z. Li, H. Wang, J. Qian, B. Peng, Y. Mao, W. Chen, and X. Yan, “Explanations from large language models make small reasoners better,” in *Proc. 2nd Workshop Sustain. AI*, 2024. [Online]. Available: <https://openreview.net/forum?id=rH8ZUcFL9r>
- [94] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhosht, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister, “Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes,” in *Proc. Findings Assoc. Comput. Linguistics, ACL*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds., Toronto, ON, Canada: Association for Computational Linguistics, 2023, pp. 8003–8017.
- [95] Y. Huang, Y. Chen, Z. Yu, and K. McKeown, “In-context learning distillation: Transferring few-shot learning ability of pre-trained language models,” 2022, *arXiv:2212.10670*.
- [96] Y. Jiang, C. Chan, M. Chen, and W. Wang, “Lion: Adversarial distillation of proprietary large language models,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, H. Bouamor, J. Pino, and K. Bali, Eds., Singapore: Association for Computational Linguistics, Dec. 2023, pp. 3134–3154. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.189>
- [97] M. Wu, A. Waheed, C. Zhang, M. Abdul-Mageed, and A. F. Aji, “LaMiniLM: A diverse herd of distilled models from large-scale instructions,” in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics*, Y. Graham and M. Purver, Eds., Mar. 2024, pp. 944–964. [Online]. Available: <https://aclanthology.org/2024.eacl-long.57>
- [98] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification,” in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2024, pp. 932–949, doi: [10.1145/3620666.3651335](https://doi.org/10.1145/3620666.3651335).
- [99] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” 2023, *arXiv:2305.13245*.
- [100] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” 2019, *arXiv:1911.02150*.
- [101] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” 2022, *arXiv:2211.17192*.
- [102] S. Prabhu, “PEDAL: Enhancing greedy decoding with large language models using diverse exemplars,” 2024, *arXiv:2408.08869*.
- [103] G. Franceschelli and M. Musolesi, “Creative beam search: LLM-as-a-Judge for improving response generation,” 2024, *arXiv:2405.00099*.
- [104] T. Kim, A. T. Suresh, K. Papineni, M. Riley, S. Kumar, and A. Benton, “Exploring and improving drafts in blockwise parallel decoding,” 2024, *arXiv:2404.09221*.



RUI LI received the bachelor's degree in communication engineering from South China University of Technology (SCUT), in 2018, and the Ph.D. degree in computer science and engineering from Sun Yat-sen University, Guangzhou, China, in 2023. He is currently a Researcher with China Telecom Research Institute, Guangzhou. His research interests include multi-access edge computing, distributed optimization, and stochastic network optimization.



DEJI FU received the bachelor's degree in computer engineering from Fudan University, in 2012. He is currently a Researcher with China Telecom Research Institute, Guangzhou. His research interests include cloud computing and AI infrastructure optimization.



CHUNYU SHI received the master's degree in computer technology from North China Electric Power University, in 2018. He is currently with the Cloud and Network Operation Technology Department, China Telecom Research Institute, Guangzhou. His research interests include cloud computing, intelligent computing, and testing.



ZHILAN HUANG received the bachelor's degree from Jilin University, Changchun, China, in 2005, and the master's degree from Sun Yat-sen University, Guangzhou, China, in 2007. She is a Senior Engineer with China Telecom Research Institute. Her research interests include cloud computing, artificial intelligence, and network communications.



GANG LU is currently a Professor-Level Senior Engineer and an Expert Committee Member with China Telecom Research Institute. His research interests include cloud computing and network communications.

...